



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 大作业-计算器的实现

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 11 月 20 日

实 验 报 告

【实验目的】

使用这个学期所学，进行一个简单的 Java 项目实践

【实验要求】

学习使用图形用户界面(GUI)设计实现一个简单的计算器，能进行 $+$ $-$ $\times$ $/$ 四则运算
(注意数字大小的越界考虑，一般计算器输入的数字个数不超过 16 个)

【软件实验流程图】

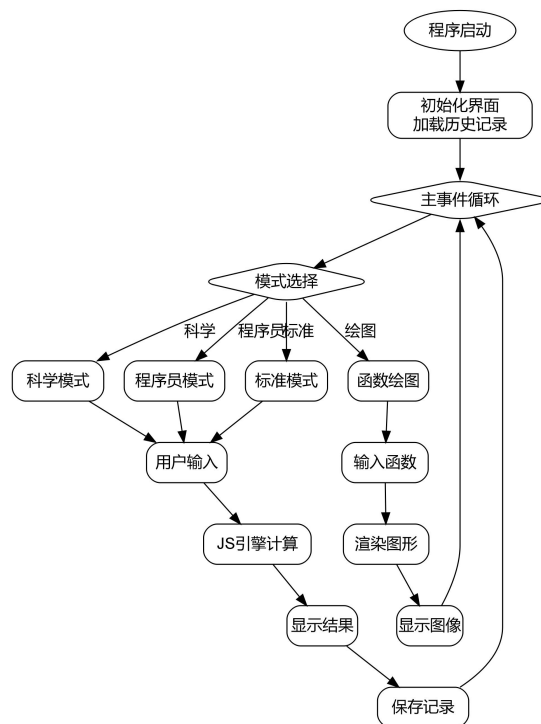


图 1 软件流程图

【页面布局设计图】



图 2 页面布局图 1

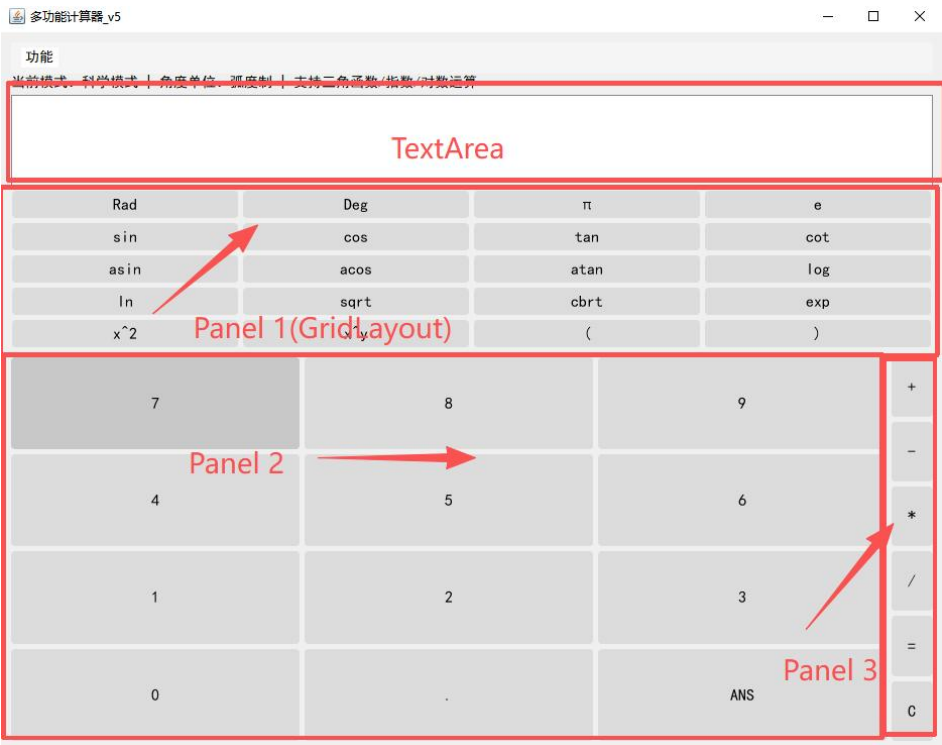


图 3 页面布局图 2



图 4 布局设计图 3

【实验具体内容】

一、实现一个简易计算器

(1) 实验源程序

```
package cn;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Calculator extends JFrame implements ActionListener {
    // 显示区域
    private JTextField displayField;
    // 按钮
    private JButton[] numberButtons;
    private JButton addButton, subButton, mulButton, divButton;
    private JButton decButton, equButton, clrButton;
    // 计算相关变量
    private String currentInput = "";
    private String operator = "";
    private double firstNumber = 0;
    private boolean isNewOperation = true;
    public Calculator() {
        // 设置窗口基本属性
        setTitle("简易计算器");
        //设置窗口大小(后面使用 pack()自动调整)
```

美观

```
//setSize(400, 500);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// 我们可以在构造函数中指定组件之间的水平和垂直间距（例如 10 像素），让界面更

setLayout(new BorderLayout(10, 10));
setResizable(false);
// --- 创建显示区域 ---
displayField = new JTextField();
displayField.setEditable(false);
displayField.setHorizontalAlignment(JTextField.RIGHT);
displayField.setFont(new Font("Arial", Font.BOLD, 20));
// displayField.setBounds(30, 20, 240, 50); // 不再需要手动设置边界
// --- 将显示区域添加到 BorderLayout 的 "North" 区域 ---
// BorderLayout 会自动拉伸 North 区域的组件以填满容器的宽度
add(displayField, BorderLayout.NORTH);
// --- 初始化所有按钮 ---
numberButtons = new JButton[10];
for (int i = 0; i < 10; i++) {
    numberButtons[i] = new JButton(String.valueOf(i));
    numberButtons[i].addActionListener(this);
    numberButtons[i].setFont(new Font("Arial", Font.BOLD, 16));
}
decButton = new JButton(".");
equButton = new JButton("=");
clrButton = new JButton("C");
addButton = new JButton("+");
subButton = new JButton("-");
mulButton = new JButton("*");
divButton = new JButton("/");
JButton[] funcButtons = {decButton, equButton, clrButton, addButton,
subButton, mulButton, divButton};
for (JButton btn : funcButtons) {
    btn.addActionListener(this);
    btn.setFont(new Font("Arial", Font.BOLD, 16));
}

// 1. 创建一个新的 JPanel 来容纳所有按钮
JPanel buttonPanel = new JPanel();
// 2. 为这个面板设置 GridLayout。我们需要 5 行 4 列的网格来容纳所有按钮。
buttonPanel.setLayout(new GridLayout(5, 4, 5, 5));
// GridLayout 会按照添加顺序，从左到右、从上到下地填充网格。（这里我们没有用
完网格）
// 我们需要一个占位符来处理 "0" 按钮占两格和 "=" 按钮占两格的情况。
// 一个简单的技巧是添加一个不可见的组件（如空的 JLabel）来占据多余的位置。
```

```

// 占位符
JLabel placeholder = new JLabel();
buttonPanel.add(clearButton);
buttonPanel.add(divButton);
buttonPanel.add(mulButton);
buttonPanel.add(subButton);
buttonPanel.add(numberButtons[7]);
buttonPanel.add(numberButtons[8]);
buttonPanel.add(numberButtons[9]);
buttonPanel.add(addButton);
buttonPanel.add(numberButtons[4]);
buttonPanel.add(numberButtons[5]);
buttonPanel.add(numberButtons[6]);
// "=" 按钮会占据第 3 行第 4 列和第 4 行第 4 列
buttonPanel.add(equButton);
buttonPanel.add(numberButtons[1]);
buttonPanel.add(numberButtons[2]);
buttonPanel.add(numberButtons[3]);
// 用占位符填充 "=" 按钮下方的空位
buttonPanel.add(placeholder);
// "0" 按钮会占据第 5 行第 1 列和第 5 行第 2 列
buttonPanel.add(numberButtons[0]);
buttonPanel.add(decButton);
// 第 5 行第 4 列是空的，GridLayout 会自动处理
// BorderLayout 会自动拉伸 Center 区域的组件以填满剩余的空间
add(buttonPanel, BorderLayout.CENTER);

// pack() 会根据所有组件的 preferred size 来计算并设置窗口的最佳大小
pack();
// 设置窗口居中显示
setLocationRelativeTo(null);
// 设置窗口可见
setVisible(true);
}

@Override
public void actionPerformed(ActionEvent e) {
    // 处理数字按钮点击
    for (int i = 0; i < 10; i++) {
        if (e.getSource() == numberButtons[i]) {
            if (isNewOperation) {
                currentInput = "";
                isNewOperation = false;
            }
            currentInput += i;
        }
    }
}

```

```

        displayField.setText(currentInput);
    }
}

// 处理小数点
if (e.getSource() == decButton) {
    if (!currentInput.contains(".")) {
        currentInput += ".";
        displayField.setText(currentInput);
    }
}

// 处理清除按钮
if (e.getSource() == clrButton) {
    currentInput = "";
    operator = "";
    firstNumber = 0;
    displayField.setText("");
    isNewOperation = true;
}

// 处理运算符
if (e.getSource() == addButton) {
    handleOperator("+");
}
if (e.getSource() == subButton) {
    handleOperator("-");
}
if (e.getSource() == mulButton) {
    handleOperator("*");
}
if (e.getSource() == divButton) {
    handleOperator("/");
}

// 处理等于按钮
if (e.getSource() == equButton) {
    calculateResult();
}
}

// 处理运算符逻辑
private void handleOperator(String op) {
    if (!currentInput.isEmpty()) {

```

```

        firstNumber = Double.parseDouble(currentInput);
        operator = op;
        isNewOperation = true;
    }
}

// 计算结果
private void calculateResult() {
    if (!currentInput.isEmpty() && !operator.isEmpty()) {
        double secondNumber = Double.parseDouble(currentInput);
        double result = 0;

        switch (operator) {
            case "+":
                result = firstNumber + secondNumber;
                break;
            case "-":
                result = firstNumber - secondNumber;
                break;
            case "*":
                result = firstNumber * secondNumber;
                break;
            case "/":
                if (secondNumber != 0) {
                    result = firstNumber / secondNumber;
                } else {
                    displayField.setText("错误");
                    currentInput = "";
                    operator = "";
                    firstNumber = 0;
                    isNewOperation = true;
                    return;
                }
                break;
        }

        // 处理整数情况，不显示小数点
        if (result == (long) result) {
            displayField.setText(String.valueOf((long) result));
        } else {
            displayField.setText(String.valueOf(result));
        }

        currentInput = String.valueOf(result);
    }
}

```

```

        operator = "";
        isNewOperation = true;
    }
}
public static void main(String[] args) {
    // 在事件调度线程中创建 GUI
    SwingUtilities.invokeLater(Calculator::new);
}
}

```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实现思路：根据软件实现的一般思路，我们可以将要完成的工作分为两类，其一为负责完成与用户直接进行交互的逻辑与功能，这里指的主要是 UI 的设计，即前端；另一类为完成用户看不到的背后逻辑部分，进行数据存储，解析与计算方面，这里主要指的是计算器功能的实现。由此，思路就很明确了，我们先使用 GUI 工具开发一个 UI，实现与用户交互，最后通过监听的功能，调用我们数据计算的代码，由于这里我们计算逻辑并不难实现，因此重点在于前端即 UI 的规划与设计

1) 使用 swing 先开发 GUI

由于简易计算器并非我们的最终的目标，为了后期进一步嵌合其他功能，我们选择了 BorderLayout 布局，我们的设想是如普通计算器一样，在顶部设置一个文本框，在中部设置一系列按键，经过一系列草图设计，参考 Windows 系统下的计算器，同时我们现在只考虑实现普通的四则运算，我们所需一个 5x4 的按键矩阵，因此，我们编写相应的代码，同时运行以验证可视化效果。

在编写过程中，我发现一个严重的问题，我想要实现的按钮不够 20 个，会在 GUI 上出现空白，非常不好看：

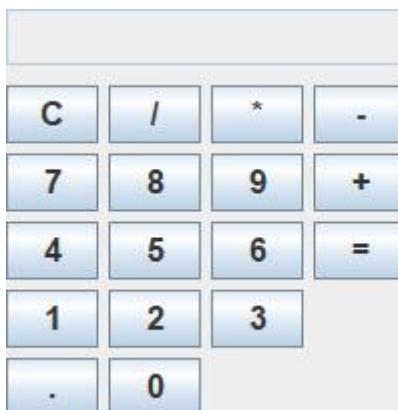


图 5 运行结果截图

因此，我们把按钮面板的布局换成 GridBagLayout，在该布局模式下，我们可以自由控制每个按钮的位置，同时可以令某一个按钮占两行，由此可以填补空白，

我们更改布局模式，由于 GridBagLayout 实现该功能的方法是通过 gbc 类与 gbl.setConstraints 方式来实现的：

```
// 简化 GridBagLayout 的按钮添加
private void addButtonToPanel(JPanel panel, GridBagLayout gbl,
GridBagConstraints gbc,
                                JButton btn, int x, int y, int width, int height)
{
    // 列坐标
    gbc.gridx = x;
    // 行坐标
    gbc.gridy = y;
    // 跨列数
    gbc.gridwidth = width;
    // 跨行数
    gbc.gridheight = height;
    gbl.setConstraints(btn, gbc);
    panel.add(btn);
}
```

调用该方法后，布局便成功修改：



图 6 简易计算器的 GUI

可以发现，我们基本实现了功能。

2) 实现基本四则运算

这里我们只需要考虑两个数据之间的运算，同时仅限于四则运算。对于输入数据，我们需要实时显示我们输入的数据，即将按键值转化为字符（循环+if 匹配），再拼接字符来显示数据，同时最后当我们点击运算符（+，-，*，/）时将字符串转化为数据来获得操作数，至于运算符也可以那样解决。在此种方法下我们不需要考

虑复杂的表达式优先级问题。由于我们采用 long 类型来存储数据结果，对于一般的数据（小于 16 位的数据）这个长度是没问题的。

输入 1+1, 检验结果：



图 7 计算逻辑验证

综上，基本的逻辑我们已经实现，但是这仅是差强人意，对于非法计算式与超长数据的异常处理我们并没有完成，同时，这个 UI 界面也透露着一股简陋，可以进一步优化，使其变得现代一些。还有，我忘记设计退格键了，因此输错数据只能一键清除，这无疑有些反人类；最后，对于复杂表达式与科学计算的任务也难以胜任，综上，我们将持续进行优化。

二、GUI 改进版+功能完善的简易计算器实现

(1) 实验源程序

```
package cn;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;

public class Caculator_v2 extends JFrame implements ActionListener {
    // ===== 核心成员变量 =====
    private JPanel mainPanel;
```

```

private JPanel northPanel;
// 顶部总面板
private JPanel topControlPanel;
// 顶部控制面板（仅显示标题）
private JPanel displayPanel;
// 输入输出显示面板
private JTextArea displayArea;
private JScrollPane displayScroll;
private String currentExpr = "";
private final int MAX_DIGIT = 16;
private ScriptEngine jsEngine;
private String ans = ""; // 上一次计算结果（ANS 功能）
// ===== 样式常量 =====
private static final Font CHINESE_FONT = new Font("黑体", Font.PLAIN, 14);
private static final Font BUTTON_FONT = new Font("黑体", Font.PLAIN, 16);
private static final Font DISPLAY_FONT = new Font("黑体", Font.BOLD, 18);
private static final Color BUTTON_BACKGROUND = new Color(220, 220, 220);
// 正常背景色
private static final Color BUTTON_HOVER_BACKGROUND = new Color(200, 200, 200);
// 悬停背景色
private static final Color BUTTON_PRESS_BACKGROUND = new Color(180, 180, 180);
// 按下背景色
private static final Color BUTTON_DISABLED_BACKGROUND = new Color(200, 200,
200);
// 禁用背景色
private static final Color BUTTON_FOREGROUND = Color.BLACK;
// 前景色（文字颜色）
private static final Color CONTROL_PANEL_BACKGROUND = new Color(245, 245, 245);
// 顶部控制面板背景色
//自定义圆角按钮（修复颜色初始化）
class RoundButton extends JButton {
// 圆角半径
private int radius = 8;
public RoundButton(String text) {
super(text);
}
// 初始化样式（关键：确保颜色在创建时就设置）
initStyle();
}
private void initStyle() {
// 1. 显式设置前景色和背景色（解决初始化未上色问题）
setForeground(BUTTON_FOREGROUND);
setBackground(BUTTON_BACKGROUND);
// 2. 基础样式配置
setFont(BUTTON_FONT);
    
```

```

// 取消默认边框
setBorderPainted(false);
setFocusPainted(true);
// 禁用默认背景填充，使用自定义绘制
setContentAreaFilled(false);
// 3. 鼠标事件监听（处理状态切换）
addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent e) {
        if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
    }
    @Override
    public void mouseExited(MouseEvent e) {
        if (isEnabled()) setBackground(BUTTON_BACKGROUND);
    }
    @Override
    public void mousePressed(MouseEvent e) {
        if (isEnabled()) setBackground(BUTTON_PRESS_BACKGROUND);
    }
    @Override
    public void mouseReleased(MouseEvent e) {
        if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
    }
});
}
// 重写绘制方法：确保背景色和前景色正确渲染
@Override
protected void paintComponent(Graphics g) {
    Graphics2D g2d = (Graphics2D) g.create();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON); // 抗锯齿
    // 绘制圆角背景（区分启用/禁用状态）
    if (isEnabled()) {
        // 使用当前状态的背景色
        g2d.setColor(getBackground());
    } else {
        // 禁用状态固定背景色
        g2d.setColor(BUTTON_DISABLED_BACKGROUND);
    }
    g2d.fillRoundRect(0, 0, getWidth(), getHeight(), radius, radius);
    // 绘制文字（确保前景色正确，且文字居中）
    g2d.setColor(getForeground()); // 使用设置的前景色
    FontMetrics fm = g2d.getFontMetrics();
    // 水平居中

```

```

        int textX = (getWidth() - fm.stringWidth(getText())) / 2;
        // 垂直居中
        int textY = (getHeight() + fm.getAscent()) / 2 - fm.getDescent();
        g2d.drawString(getText(), textX, textY);
        g2d.dispose();
    }
    // 可选：设置圆角半径
    public void setRadius(int radius) {
        this.radius = radius;
        repaint();
    }
}
// 构造方法
public Caculator_v2() {
    setTitle("基础功能计算器");
    setSize(600, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setResizable(true);
    // 初始化脚本引擎
    jsEngine = new ScriptEngineManager().getEngineByName("JavaScript");
    // 初始化界面布局和组件
    initLayout();
    initComponents();
    // 显示窗口
    setVisible(true);
}
// 布局初始化
private void initLayout() {
    // 主面板（边框布局，带边距）
    mainPanel = new JPanel(new BorderLayout(10, 10));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
    add(mainPanel);
    // 顶部总面板（垂直排列：控制面板 + 显示面板）
    northPanel = new JPanel(new BorderLayout(5, 5));
    // 顶部控制面板（仅显示标题）
    topControlPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 5));
    topControlPanel.setBackground(CONTROL_PANEL_BACKGROUND);
    topControlPanel.setPreferredSize(new Dimension(580, 35));
    // 显示面板（垂直排列：提示标签 + 文本区域）
    displayPanel = new JPanel(new BorderLayout(5, 5));
    // 组装顶部面板
    northPanel.add(topControlPanel, BorderLayout.NORTH);
    northPanel.add(displayPanel, BorderLayout.SOUTH);
}

```

```

        // 组装主面板
        mainPanel.add(northPanel, BorderLayout.NORTH);
        // 直接添加标准模式面板
        mainPanel.add(new StandardModePanel(), BorderLayout.CENTER);
    }
    // 组件初始化
    private void initComponents() {
        // 1. 顶部标题
        JLabel titleLabel = new JLabel("基础计算器（支持加减乘除、百分比、倒数、平方根等）");
        titleLabel.setFont(CHINESE_FONT);
        topControlPanel.add(titleLabel);
        // 2. 显示区域组件
        JLabel tipLabel = new JLabel("提示：输入表达式后按等号计算，支持ANS复用上次结果");
        tipLabel.setFont(CHINESE_FONT);
        displayPanel.add(tipLabel, BorderLayout.NORTH);
        displayArea = new JTextArea(3, 30);
        displayArea.setFont(DISPLAY_FONT);
        displayArea.setEditable(false);
        displayArea.setLineWrap(true);
        displayArea.setBackground(new Color(240, 240, 240));
        displayArea.setForeground(Color.BLACK);
        displayScroll = new JScrollPane(displayArea);
        displayPanel.add(displayScroll, BorderLayout.CENTER);
    }
    // 运算逻辑
    // 格式化计算结果（去除末尾.0，过长用科学计数法）
    private String formatResult(String result) {
        if (result.endsWith(".0")) {
            return result.substring(0, result.length() - 2);
        }
        if (result.length() > MAX_DIGIT && !result.contains("e")) {
            return new java.math.BigDecimal(result).toEngineeringString();
        }
        return result;
    }
    // 获取表达式中最后一个数字（用于正负号切换）
    private String getLastNumber(String expr) {
        StringBuilder num = new StringBuilder();
        for (int i = expr.length() - 1; i >= 0; i--) {
            char c = expr.charAt(i);
            if (Character.isDigit(c) || c == '.' || c == '-') {
                num.insert(0, c);
            }
        }
    }

```

```

        } else {
            break;
        }
    }
    return num.toString();
}
// 按钮点击事件处理
@Override
public void actionPerformed(ActionEvent e) {
    String btnText = ((JButton) e.getSource()).getText();
    handleStandardMode(btnText);
}
//标准模式事件处理（基础功能核心）
private void handleStandardMode(String btnText) {
    switch (btnText) {
        case "C": case "CE":
            currentExpr = "";
            displayArea.setText("");
            break;
        case "DEL":
            if (!currentExpr.isEmpty()) {
                currentExpr = currentExpr.substring(0, currentExpr.length() -
1);
                displayArea.setText(currentExpr);
            }
            break;
        case "=":
            calculateResult();
            break;
        case "1/x":
            // 倒数
            currentExpr += "1/";
            displayArea.setText(currentExpr);
            break;
        case "√":
            // 平方根
            currentExpr += "Math.sqrt(";
            displayArea.setText(currentExpr);
            break;
        case "ANS":
            // 上一次结果
            if (!ans.isEmpty()) {
                currentExpr += ans;
                displayArea.setText(currentExpr);
            }
    }
}

```

```

        }
        break;
    case "+/-":
        // 正负号切换
        String lastNum = getLastNumber(currentExpr);
        if (!lastNum.isEmpty()) {
            int lastNumStart = currentExpr.length() - lastNum.length();
            String sign = lastNum.startsWith("-") ? "" : "-";
            currentExpr = currentExpr.substring(0, lastNumStart) + sign +
lastNum.replaceFirst("-", "");
            displayArea.setText(currentExpr);
        }
        break;
    case "%":
        // 百分比
        currentExpr += "/100";
        displayArea.setText(currentExpr);
        break;
    default:
        // 数字和运算符
        if (btnText.matches("[0-9]") &&
getLastNumber(currentExpr).length() >= MAX_DIGIT) {
            JOptionPane.showMessageDialog(this, "数字长度不能超过 16 位!",
"警告", JOptionPane.WARNING_MESSAGE);
            return;
        }
        currentExpr += btnText;
        displayArea.setText(currentExpr);
    }
}

//执行计算并显示结果（基础功能核心）

private void calculateResult() {
    if (currentExpr.isEmpty()) return;
    try {
        String originalExpr = currentExpr;
        String expr = currentExpr;
        // 执行计算（基础运算，无需角度转换等）
        Object result = jsEngine.eval(expr);
        String resultStr = formatResult(result.toString());
        String fullExpr = originalExpr + "=\n" + resultStr;
        displayArea.setText(fullExpr);

        // 更新上一次结果（ANS）
    }
}

```

```

        currentExpr = resultStr;
        ans = resultStr;
    } catch (ScriptException e) {
        displayArea.setText("表达式错误! ");
        currentExpr = "";
    }
}

//标准模式面板（基础功能按钮）
class StandardModePanel extends JPanel {
    public StandardModePanel() {
        setLayout(new GridLayout(6, 5, 5, 5));
        String[] standardBtns = {
            "MC", "MS", "M+", "M-", "DEL", // 记忆功能保留（基础常用功能）
            "7", "8", "9", "/", "%",
            "4", "5", "6", "*", "1/x",
            "1", "2", "3", "-", "√",
            "0", ".", "ANS", "+", "=",
            "C", "CE", "+/-", "(", ")"
        };
        for (String text : standardBtns) {
            RoundButton btn = new RoundButton(text);
            btn.addActionListener(Caculator_v2.this);
            add(btn);
        }
    }
}

// 主方法
public static void main(String[] args) {
    SwingUtilities.invokeLater(Caculator_v2::new);
}
}

```

1) 在原来代码的基础上继续优化 UI 界面

学习学长的优秀作品，我终于发现我的简陋风从何而来：

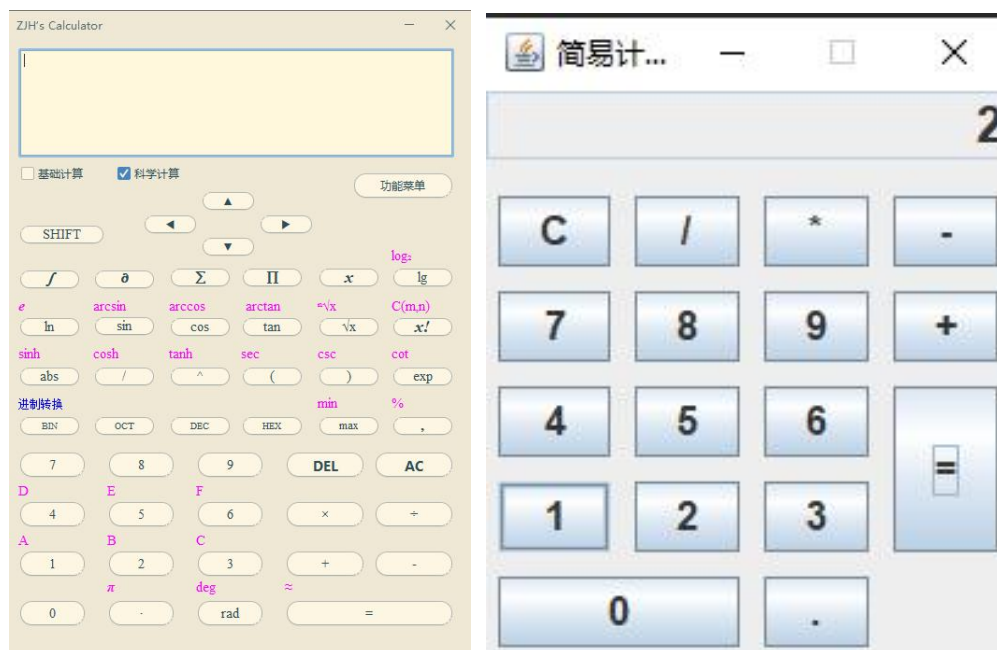


图 8 优美的 UI 设计与现状对比

其一，我的 UI 的按钮样式过于单一，其二，学长的配色丰富，视觉效果非常好，其三，我的输入框不够明显，没有凸显边框，因而显得简陋。

从第一步开始，我们需要修改固定的按钮样式，由于 JButton 默认样式为矩形，但是它也支持对它进行继承重写以修改按钮样式，因此我们需要重头开始设计按钮，经过搜索，我们找到了方法，那就是使用 Graphics2D 类来绘制，`g2d.fillRoundRect(0, 0, getWidth(), getHeight(), radius, radius)`便可以将其变为圆角矩形，同时我们为其添加颜色，运行结果：



图 9 新的按钮样式

看起来不错，在此基础上，我想实现当鼠标悬停在按钮上时，按钮颜色变深的效果，就如 Windows 系统下一样，为实现这个功能，我们需要监听鼠标的活动，经过上网学习后，参考一位大神的代码，我们实现了 `MouseListener` 接口，但是完全实现它需要我们重写全部 5 个方法，但是我们只需要监听 4 个事件，因此我们 new 了一个内部匿名类 `MouseAdapter`，其提供了全部的 5 个方法的空实现，因此我们只需要实现我们需要的方法：

```
addMouseListener(new MouseAdapter() {
```

```

        @Override
        //悬空
        public void mouseEntered(MouseEvent e) {
            if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
        }
        // 离开
        @Override
        public void mouseExited(MouseEvent e) {
            if (isEnabled()) setBackground(BUTTON_BACKGROUND);
        }
        //按下
        @Override
        public void mousePressed(MouseEvent e) {
            if (isEnabled()) setBackground(BUTTON_PRESS_BACKGROUND);
        }
        //松开
        @Override
        public void mouseReleased(MouseEvent e) {
            if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
        }
    });
}
    
```

每当对应事件被监听到，对应的按钮背景色就会变深，同时，为了凸显输入界面，我们修改其的背景色，且改变了整体的背景色，经历这一系列修改，我们运行下看下结果：



图 10 悬浮变色功能实现

如此，我们的计算器的 UI 界面可以称得上优雅了。

2) 数据处理逻辑的补完

在简易计算器的实现中，我们做了一些反人类的设计，因此我们需要补上这些漏洞，比如退格键的实现，支持整个运算式的输入与计算，对于数据处理的后端部分，这无异于推倒重做，考虑到我们后来的功能的实现要依靠这个，我还是完全重写了数据处理部分的代码。

如何处理每个数据运算符的优先级，这我们在数据结构中学过，即通过栈结构来实现中缀表达式转为后缀表达式，后进行处理计算，我原以为要我从零开始实现这个算法，心里有点发怵，后来我发现我们完全可以基于 JS 引擎进行计算开发，通过 `ScriptEngine.eval()` 让 JS 引擎内部自动处理表达式解析，代码可以非常简单（**暂时看来是这样，后来发现并不是**）。

```
private void calculateResult() {
    if (currentExpr.isEmpty()) return;
    try {
        String originalExpr = currentExpr;
        String expr = currentExpr;

        // 执行计算（基础运算，无需角度转换等）
        Object result = jsEngine.eval(expr);
        String resultStr = formatResult(result.toString());
        String fullExpr = originalExpr + "=\n" + resultStr;
        displayArea.setText(fullExpr);

        // 更新上一次结果（ANS）
        currentExpr = resultStr;
        ans = resultStr;
    } catch (ScriptException e) {
        displayArea.setText("表达式错误！");
        currentExpr = "";
    }
}
```

只需要将表达式转为字符串，运算便可以自动实现，其他的逻辑几乎不变，仅需要实现不同的按键的功能，如退格，即对字符串进行切片处理即可，其他功能如此好实现，仅是对应的 switch 分支多了一点罢了。完善逻辑的编写后，我们进行验证计算逻辑是否正确：



图 11 测试计算式输入



图 12 结果验证

可以发现，我们的结果是正确的，功能实现得是比较成功的，因此可以进一步添加功能了。

三、实现历史记录功能

(1) 实验源代码

```
package cn;
```

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.io.*;
import java.text.SimpleDateFormat;
import java.util.Date;
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;
public class Caculate_v3 extends JFrame implements ActionListener {
    // 核心成员变量（新增历史记录相关）
    private JPanel mainPanel;
    // 顶部总面板
    private JPanel northPanel;
    // 顶部控制面板（新增菜单板）
    private JPanel topControlPanel;
    // 输入输出显示面板
    private JPanel displayPanel;
    private JTextArea displayArea;
    private JScrollPane displayScroll;
    private String currentExpr = "";
    private final int MAX_DIGIT = 16;
    private ScriptEngine jsEngine;
    // 上一次计算结果（ANS 功能）
    private String ans = "";
    // 历史记录核心组件
    // 历史记录数据模型
    private DefaultListModel<String> historyModel;
    // 历史记录列表组件
    private JList<String> historyList;
    // 格式化时间戳
    private SimpleDateFormat dateFormat;
    // 历史记录存储文件
    private static final String HISTORY_FILE = "calc_history.txt";
    // 样式常量（菜单颜色配置）
    private static final Font CHINESE_FONT = new Font("黑体", Font.PLAIN, 14);
    private static final Font BUTTON_FONT = new Font("黑体", Font.PLAIN, 16);
    private static final Font DISPLAY_FONT = new Font("黑体", Font.BOLD, 18);
    private static final Font LIST_FONT = new Font("黑体", Font.PLAIN, 13); // 历史列表字体
    private static final Color BUTTON_BACKGROUND = new Color(220, 220, 220);

```

```

private static final Color BUTTON_HOVER_BACKGROUND = new Color(200, 200, 200);
private static final Color BUTTON_PRESS_BACKGROUND = new Color(180, 180, 180);
private static final Color BUTTON_DISABLED_BACKGROUND = new Color(200, 200,
200);
private static final Color BUTTON_FOREGROUND = Color.BLACK;
private static final Color CONTROL_PANEL_BACKGROUND = new Color(245, 245, 245);
private static final Color MENU_BACKGROUND = CONTROL_PANEL_BACKGROUND; // 菜
单与控制面板颜色统一
private static final Color MENU_FOREGROUND = Color.BLACK;
private static final Color MENU_SELECTED_BACKGROUND = new Color(200, 200, 200);
private static final Color MENU_SELECTED_FOREGROUND = Color.BLACK;
// 自定义圆角按钮（复用版本 1）
class RoundButton extends JButton {
    private int radius = 8;
    public RoundButton(String text) {
        super(text);
        initStyle();
    }
    private void initStyle() {
        setForeground(BUTTON_FOREGROUND);
        setBackground(BUTTON_BACKGROUND);
        setFont(BUTTON_FONT);
        setBorderPainted(false);
        setFocusPainted(true);
        setContentAreaFilled(false);
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
            }
            @Override
            public void mouseExited(MouseEvent e) {
                if (isEnabled()) setBackground(BUTTON_BACKGROUND);
            }
            @Override
            public void mousePressed(MouseEvent e) {
                if (isEnabled()) setBackground(BUTTON_PRESS_BACKGROUND);
            }
            @Override
            public void mouseReleased(MouseEvent e) {
                if (isEnabled()) setBackground(BUTTON_HOVER_BACKGROUND);
            }
        });
    }
}
    
```

```

@Override
protected void paintComponent(Graphics g) {
    Graphics2D g2d = (Graphics2D) g.create();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
    if (isEnabled()) {
        g2d.setColor(getBackground());
    } else {
        g2d.setColor(BUTTON_DISABLED_BACKGROUND);
    }
    g2d.fillRoundRect(0, 0, getWidth(), getHeight(), radius, radius);
    g2d.setColor(getForeground());
    FontMetrics fm = g2d.getFontMetrics();
    int textX = (getWidth() - fm.stringWidth(getText())) / 2;
    int textY = (getHeight() + fm.getAscent()) / 2 - fm.getDescent();
    g2d.drawString(getText(), textX, textY);
    g2d.dispose();
}

public void setRadius(int radius) {
    this.radius = radius;
    repaint();
}
}

//构造方法（新增历史记录初始化）
public Caculate_v3() {
    setTitle("基础计算器");
    setSize(600, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setResizable(true);
    // 初始化脚本引擎
    jsEngine = new ScriptEngineManager().getEngineByName("JavaScript");
    // 初始化历史记录组件（核心新增）
    historyModel = new DefaultListModel<>();
    historyList = new JList<>(historyModel);
    historyList.setFont(LIST_FONT);
    historyList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    // 时间戳格式：时:分:秒
    dateFormat = new SimpleDateFormat("HH:mm:ss");
    loadHistory(); // 启动时加载本地历史记录
    initHistoryListDoubleClick(); // 初始化双击复用功能
    // 初始化界面布局和组件
    initLayout();
    initComponents();
}
    
```

```

// 新增菜单事件监听
initMenuListeners();
// 显示窗口
setVisible(true);
}
// 布局初始化（新增菜单面板）
private void initLayout() {
    mainPanel = new JPanel(new BorderLayout(10, 10));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
    add(mainPanel);
    northPanel = new JPanel(new BorderLayout(5, 5));
    // 顶部控制面板：改为左侧菜单 + 右侧标题（新增菜单）
    topControlPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
    topControlPanel.setBackground(CONTROL_PANEL_BACKGROUND);
    topControlPanel.setPreferredSize(new Dimension(580, 35));
    displayPanel = new JPanel(new BorderLayout(5, 5));
    // 组装顶部面板
    northPanel.add(topControlPanel, BorderLayout.NORTH);
    northPanel.add(displayPanel, BorderLayout.SOUTH);
    mainPanel.add(northPanel, BorderLayout.NORTH);
    mainPanel.add(new StandardModePanel(), BorderLayout.CENTER);
}
// 组件初始化
private void initComponents() {
    // 1. 顶部菜单
    JMenuBar menuBar = new JMenuBar();
    menuBar.setBackground(MENU_BACKGROUND);
    menuBar.setBorderPainted(false);
    JMenu funcMenu = new JMenu("功能");
    funcMenu.setFont(CHINESE_FONT);
    funcMenu.setBackground(MENU_BACKGROUND);
    funcMenu.setForeground(MENU_FOREGROUND);
    funcMenu.setBorderPainted(false);
    // 历史记录菜单项
    JMenuItem historyItem = new JMenuItem("查看历史记录");
    historyItem.setFont(CHINESE_FONT);
    historyItem.setBackground(MENU_BACKGROUND);
    historyItem.setForeground(MENU_FOREGROUND);
    historyItem.setBorderPainted(false);
    // 清空历史记录菜单项
    JMenuItem clearHistoryItem = new JMenuItem("清空历史记录");
    clearHistoryItem.setFont(CHINESE_FONT);
    clearHistoryItem.setBackground(MENU_BACKGROUND);
    clearHistoryItem.setForeground(MENU_FOREGROUND);
}

```

```

        clearHistoryItem.setBorderPainted(false);
        // 菜单样式统一（选中状态）
        UIManager.put("MenuItem.selectionBackground",
MENU_SELECTED_BACKGROUND);
        UIManager.put("MenuItem.selectionForeground",
MENU_SELECTED_FOREGROUND);
        funcMenu.add(historyItem);
        funcMenu.addSeparator();
        funcMenu.add(clearHistoryItem);
        menuBar.add(funcMenu);
        topControlPanel.add(menuBar);
        // 2. 右侧标题
        JLabel titleLabel = new JLabel("基础计算器");
        titleLabel.setFont(CHINESE_FONT);
        topControlPanel.add(titleLabel);
        // 3. 显示区域
        JLabel tipLabel = new JLabel("提示：输入表达式后按等号计算，双击历史记录可复
用表达式");
        tipLabel.setFont(CHINESE_FONT);
        displayPanel.add(tipLabel, BorderLayout.NORTH);
        displayArea = new JTextArea(3, 30);
        displayArea.setFont(DISPLAY_FONT);
        displayArea.setEditable(false);
        displayArea.setLineWrap(true);
        displayArea.setBackground(new Color(240, 240, 240));
        displayArea.setForeground(Color.BLACK);
        displayScroll = new JScrollPane(displayArea);
        displayPanel.add(displayScroll, BorderLayout.CENTER);
    }
    //历史记录核心方法（新增）
    /**
     * 初始化历史记录双击复用：双击历史项，将表达式回填到输入框
     */
    private void initHistoryListDoubleClick() {
        historyList.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                if (e.getClickCount() == 2) { // 双击事件
                    int selectedIndex = historyList.getSelectedIndex();
                    if (selectedIndex != -1) {
                        String selectedItem =
historyModel.getElementAt(selectedIndex);
                        // 解析历史记录格式："HH:mm:ss | 表达式 = 结果" -> 提取表达
式

```

```

        String expr = selectedItem.split(" \\| ")[1].split(" =
    ")[0];

        currentExpr = expr;
        displayArea.setText(currentExpr);
    }
}

});
}

/**
 * 加载本地历史记录（程序启动时调用）
 */
private void loadHistory() {
    File historyFile = new File(HISTORY_FILE);
    if (!historyFile.exists()) return; // 文件不存在则不加载
    try (BufferedReader reader = new BufferedReader(new
FileReader(historyFile))) {
        String line;
        while ((line = reader.readLine()) != null) {
            historyModel.addElement(line); // 添加到内存模型
        }
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "加载历史记录失败: " +
e.getMessage(), "错误", JOptionPane.ERROR_MESSAGE);
    }
}

//保存历史记录到本地文件（计算完成后调用）
private void saveHistory(String record) {
    try (PrintWriter writer = new PrintWriter(new FileWriter(HISTORY_FILE,
true))) {
        writer.println(record); // 追加写入文件
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "保存历史记录失败: " +
e.getMessage(), "错误", JOptionPane.ERROR_MESSAGE);
    }
}

// 显示历史记录窗口（点击菜单时调用）

private void showHistoryWindow() {
    JFrame historyFrame = new JFrame("计算历史记录");
    historyFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    historyFrame.setSize(400, 300);
    historyFrame.setLocationRelativeTo(this);
    JPanel historyPanel = new JPanel(new BorderLayout(5, 5));

```

```

        JLabel titleLabel = new JLabel("历史记录（时间 | 表达式 = 结果）");
        titleLabel.setFont(CHINESE_FONT);
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        historyPanel.add(titleLabel, BorderLayout.NORTH);
        // 空历史记录提示
        if (historyModel.isEmpty()) {
            JLabel emptyLabel = new JLabel("暂无计算历史记录");
            emptyLabel.setFont(CHINESE_FONT);
            emptyLabel.setHorizontalAlignment(JLabel.CENTER);
            historyPanel.add(emptyLabel, BorderLayout.CENTER);
        } else {
            JScrollPane scrollPane = new JScrollPane(historyList);
            scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
            historyPanel.add(scrollPane, BorderLayout.CENTER);
        }
        historyFrame.add(historyPanel);
        historyFrame.setVisible(true);
    }
    // 清空历史记录（点击菜单时调用）

    private void clearHistory() {
        int confirm = JOptionPane.showConfirmDialog(
            this, "确定要清空所有历史记录吗？（本地文件也会删除）", "确认清空",
            JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE
        );
        if (confirm == JOptionPane.YES_OPTION) {
            historyModel.clear(); // 清空内存模型
            new File(HISTORY_FILE).delete(); // 删除本地文件
            JOptionPane.showMessageDialog(this, "历史记录已清空！", "提示",
                JOptionPane.INFORMATION_MESSAGE);
        }
    }
    // 菜单事件监听（新增）
    private void initMenuListeners() {
        JMenuBar menuBar = (JMenuBar) topControlPanel.getComponent(0);
        JMenu funcMenu = (JMenu) menuBar.getComponent(0);
        // 遍历菜单项，绑定事件
        for (Component comp : funcMenu.getMenuComponents()) {
            if (comp instanceof JMenuItem) {
                JMenuItem menuItem = (JMenuItem) comp;
                menuItem.addActionListener(e -> {
                    String itemText = menuItem.getText();
                    if (itemText.equals("查看历史记录")) {

```

```

        showHistoryWindow();
    } else if (itemText.equals("清空历史记录")) {
        clearHistory();
    }
});
}
}
}
// 工具方法
private String formatResult(String result) {
    if (result.endsWith(".0")) {
        return result.substring(0, result.length() - 2);
    }
    if (result.length() > MAX_DIGIT && !result.contains("e")) {
        return new java.math.BigDecimal(result).toEngineeringString();
    }
    return result;
}
private String getLastNumber(String expr) {
    StringBuilder num = new StringBuilder();
    for (int i = expr.length() - 1; i >= 0; i--) {
        char c = expr.charAt(i);
        if (Character.isDigit(c) || c == '.' || c == '-') {
            num.insert(0, c);
        } else {
            break;
        }
    }
    return num.toString();
}
// 按钮点击事件处理
@Override
public void actionPerformed(ActionEvent e) {
    String btnText = ((JButton) e.getSource()).getText();
    handleStandardMode(btnText);
}
private void handleStandardMode(String btnText) {
    switch (btnText) {
        case "C": case "CE":
            currentExpr = "";
            displayArea.setText("");
            break;
        case "DEL":
            if (!currentExpr.isEmpty()) {

```

```

        currentExpr = currentExpr.substring(0, currentExpr.length() -
1);

        displayArea.setText(currentExpr);
    }
    break;
case "=":
    calculateResult(); // 计算结果时自动保存历史
    break;
case "1/x":
    currentExpr += "1/";
    displayArea.setText(currentExpr);
    break;
case "√":
    currentExpr += "Math.sqrt(";
    displayArea.setText(currentExpr);
    break;
case "ANS":
    if (!ans.isEmpty()) {
        currentExpr += ans;
        displayArea.setText(currentExpr);
    }
    break;
case "+/-":
    String lastNum = getLastNumber(currentExpr);
    if (!lastNum.isEmpty()) {
        int lastNumStart = currentExpr.length() - lastNum.length();
        String sign = lastNum.startsWith("-") ? "" : "-";
        currentExpr = currentExpr.substring(0, lastNumStart) + sign +
lastNum.replaceFirst("-", "");
        displayArea.setText(currentExpr);
    }
    break;
case "%":
    currentExpr += "/100";
    displayArea.setText(currentExpr);
    break;
default:
    if (btnText.matches("[0-9]") &&
getLastNumber(currentExpr).length() >= MAX_DIGIT) {
        JOptionPane.showMessageDialog(this, "数字长度不能超过 16 位!",
"警告", JOptionPane.WARNING_MESSAGE);
        return;
    }
    currentExpr += btnText;

```

```

        displayArea.setText(currentExpr);
    }
}
// 计算方法（新增历史记录保存逻辑）
private void calculateResult() {
    if (currentExpr.isEmpty()) return;
    try {
        String originalExpr = currentExpr;
        String expr = currentExpr;
        // 执行计算
        Object result = jsEngine.eval(expr);
        String resultStr = formatResult(result.toString());
        String fullExpr = originalExpr + "=\n" + resultStr;
        displayArea.setText(fullExpr);
        // 保存历史记录到内存和本地文件
        String timeStamp = dateFormat.format(new Date());
        String historyRecord = timeStamp + " | " + originalExpr + " = " +
resultStr;
        // 内存中添加

        historyModel.addElement(historyRecord);
        // 本地文件中保存
        saveHistory(historyRecord);
        // 更新 ANS 结果
        currentExpr = resultStr;
        ans = resultStr;
    } catch (ScriptException e) {
        displayArea.setText("表达式错误！");
        currentExpr = "";
    }
}
// 标准模式面板
class StandardModePanel extends JPanel {
    public StandardModePanel() {
        setLayout(new GridLayout(6, 5, 5, 5));
        String[] standardBtns = {
            "MC", "MS", "M+", "M-", "DEL",
            "7", "8", "9", "/", "%",
            "4", "5", "6", "*", "1/x",
            "1", "2", "3", "-", "√",
            "0", ".", "ANS", "+", "=",
            "C", "CE", "+/-", "(", ")"
        };
    }
    for (String text : standardBtns) {

```

```

        RoundButton btn = new RoundButton(text);
        btn.addActionListener(Caculate_v3.this);
        add(btn);
    }
}
// 主方法
public static void main(String[] args) {
    SwingUtilities.invokeLater(Caculate_v3::new);
}
}

```

(2) 实验过程

对于历史记录的实现，参考网上一开源 Java 计算器项目，我们可以使用 `JList` 类与其专用的动态数组 `DefaultListModel<String>`，其如链表一般支持实时的增删查改，同时我们需要本地的文件对其进行读写以实现永久性的保存功能，最后在原项目的基础上，我认为需要加上双击历史记录可复用结果的功能，因为这样会进一步优化用户体验。

综上，我们需要实现的历史记录的功能为支持查看，增删，复用历史记录，现在我们一个个进行实现。



图 13 需要新添加的后端逻辑

1) 实现历史记录查看与清除

对于 UI 界面来说，我们需要添加一个菜单界面，以帮助我们进行历史记录的查看面板的查看以及清除历史记录功能的交互，这个很好实现，我们直接将

菜单添加至顶部，再一个添加菜单组件即可，在事件监听时，我们并没有事先为他们绑定监听器，而是在事后调用 `initMenuListeners` 方法，遍历菜单组件，为其一一添加监听器，因此这个写法更加动态，方便我们后续的进一步扩展。

编写完相应的代码后，我们来观察一下效果：



图 14 面板添加功能验证

我们同时要为点击查看历史记录的监听器时，其所呼出的历史记录面板进行 UI 构建，这里我们使用 `JPanel`，方便我们进行滚轮控制来查看历史记录，同时再处理好没有历史记录时的异常处理即可。简单编写后，运行效果如下：

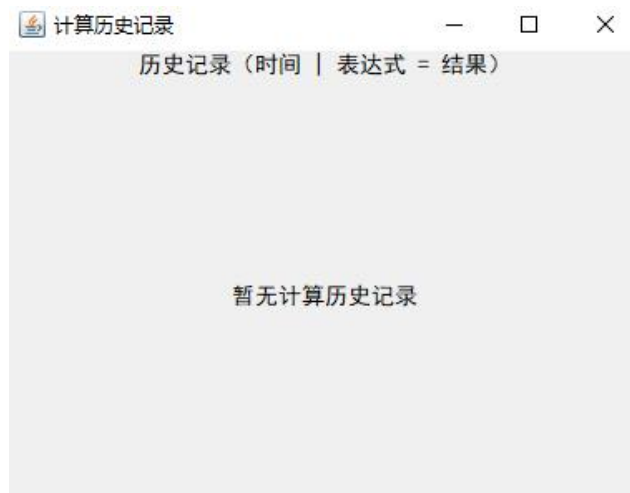


图 15 历史记录面板

现在，我们的前端逻辑构建已经基本完成了。

2) 添加历史记录逻辑

1. 计算成功自动保存结果

这个功能很好实现，直接在计算逻辑中添加即可，在事先约定好历史记录的保存格式为（时间 | 表达式 = 结果），实时添加 JList 表同时将其添加到历史记录面板上，最后将该条结果写入文件中（这里使用 FileWriter 类实现）。

2. 启动时初始化历史记录

与上次写逻辑相对，我们需要进行读，即我们一行行将表达式读入，将其一一转化为内部的数据存储格式，同时注意异常处理，我们的历史记录程序也做完了

3. 双击复用历史结果

通过给历史记录列表（JList）绑定鼠标点击监听器，判断点击次数为 2 次（双击）时，获取选中的历史记录项，按“时间戳 | 表达式 = 结果”的固定格式拆分字符串提取出表达式部分，再将其赋值给计算器的当前表达式变量（currentExpr）并更新显示区域，从而实现双击历史记录复用表达式的效果。

运行程序以验证逻辑是否正确：

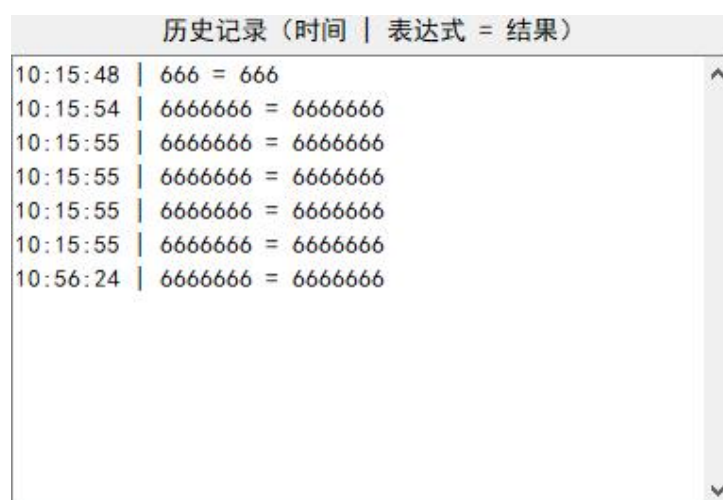


图 16 验证添加逻辑是否正确

综上，我们可以说我们的历史记录功能已经趋于完善

四、科学计算与程序员计算面板功能开发

实验过程

1) UI 设计部分

思路：仿造标准模式的实现过程，我们同样加上科学模式与程序员模式的实现，以原来的基础模式的面板作为指导，进行对程序员模式与科学模式的 UI 设计，在前面基础模式的开发经验下，这些修改已经对我来说是轻车就熟，经过一系列的修改，我们很快得到了我们想要的 UI 设计：



图 17 科学模式的 UI

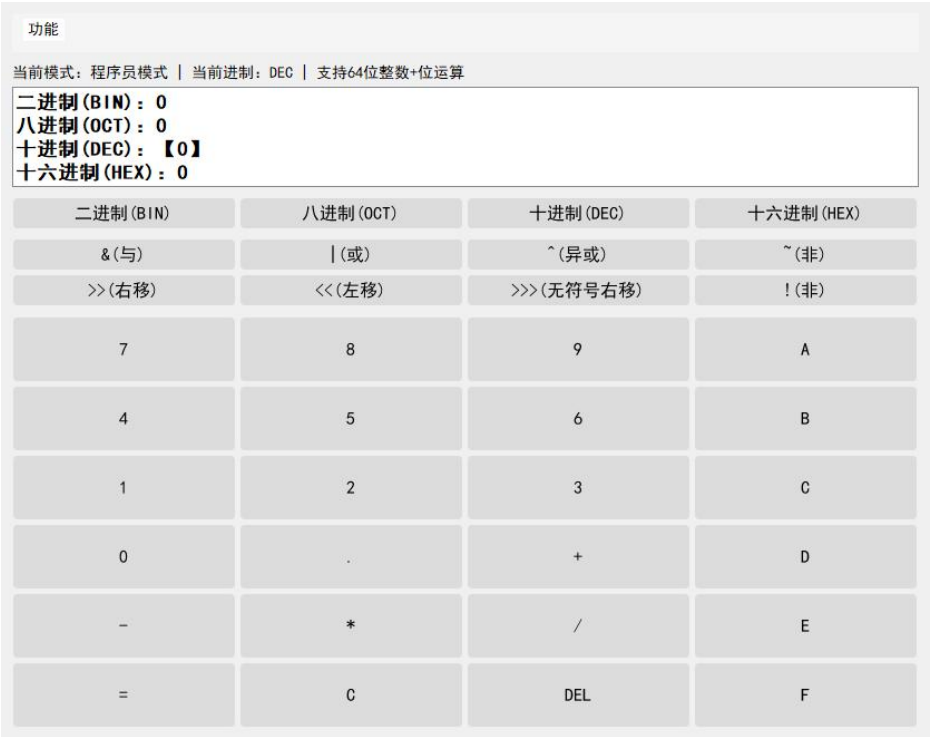


图 18 程序员模式下的 UI

3) 数据处理逻辑的添加

1. 科学模式的开发

对于我来说，这两项功能的最困难的部分并不在于设计 UI，而在于对于数据运算的处理部分。一开始，我为如何实现更高级的运算而苦恼，后面我了解到，在 JS 引擎中，其也支持高级的函数，只不过需要我们对于其表达式需要转化为引擎支持的表达式，因此，我们着手开始进行实现。

```
private void handleScientificMode(String btnText) {
    switch (btnText) {
        case "C": case "CE":
            currentExpr = "";
            displayArea.setText("");
            break;
        case "DEL":
            if (!currentExpr.isEmpty()) {
                currentExpr = currentExpr.substring(0, currentExpr.length() - 1);
                displayArea.setText(currentExpr);
            }
            break;
        case "=":
            calculateResult();
            break;
    }
}
```

```

        case "Rad":
            isRadian = true;
            JOptionPane.showMessageDialog(this, "已切换为弧度制", "提示",
JOptionPane.INFORMATION_MESSAGE);
            break;
        case "Deg":
            isRadian = false;
            JOptionPane.showMessageDialog(this, "已切换为角度制", "提示",
JOptionPane.INFORMATION_MESSAGE);
            break;
        case "π":
            currentExpr += Math.PI;
            displayArea.setText(currentExpr);
            break;
        case "e":
            currentExpr += Math.E;
            displayArea.setText(currentExpr);
            break;
        case "sin":
            wrapFunction("Math.sin");
            break;
        case "cos":
            wrapFunction("Math.cos");
            break;
        case "tan":
            wrapFunction("Math.tan");
            break;
        case "cot":
            wrapFunction("1/Math.tan");
            break;
        case "asin":
            wrapFunction("Math.asin");
            break;
        case "acos":
            wrapFunction("Math.acos");
            break;
        case "atan":
            wrapFunction("Math.atan");
            break;
        case "log":
            wrapFunction("Math.log10");
            break;
        case "ln":
            wrapFunction("Math.log");
    
```

```

        break;
    case "sqrt":
        wrapFunction("Math.sqrt");
        break;
    case "cbrt":
        wrapFunction("Math.cbrt");
        break;
    case "exp":
        wrapFunction("Math.exp");
        break;
    case "x^2":
        currentExpr += "**2";
        displayArea.setText(currentExpr);
        break;
    case "x^y":
        currentExpr += "**";
        displayArea.setText(currentExpr);
        break;
    default:
        currentExpr += btnText;
        displayArea.setText(currentExpr);
    }
}

```

如上，我们将对应的表达式都映射到了为 JS 引擎支持的形式，原有逻辑不变，我们将拼接得到的字符串送到 JS 引擎中，后返回计算结果，但是，科学模式还应处理在角度制与弧度制切换问题，我一开始想在计算逻辑里直接将弧度制结果替换为角度制结果，但是导致了很大的问题：

当我切换为角度制时，我输入任意合法表达式时，它均会报错。



图 19 角度转换出现错误

回头看，我发现了问题所在：

```
if ("科学模式".equals(currentMode) && !isRadian) {
    expr = expr.replaceAll("sin\\((.*)\\)",
        "Math.sin(Math.toRadians($1))")
        .replaceAll("cos\\((.*)\\)",
        "Math.cos(Math.toRadians($1))")
        .replaceAll("tan\\((.*)\\)",
        "Math.tan(Math.toRadians($1))")
        .replaceAll("cot\\((.*)\\)",
        "1/Math.tan(Math.toRadians($1))")
        .replaceAll("asin\\((.*)\\)",
        "Math.asin(Math.toRadians($1))")
        .replaceAll("acos\\((.*)\\)",
        "Math.acos(Math.toRadians($1))")
        .replaceAll("atan\\((.*)\\)",
        "Math.atan(Math.toRadians($1))");
}
```

在 JS 中，是不支持 `toRadians` 这样的的语句中，当我们按下按键后，所有的结果均会报错，因此，我们只能利用监听用户按下右括号的按键时，为其手动补上对应的转化逻辑：

```
case ")":
    if (needCompleteAngleFormula && currentExpr.endsWith("(")) {
```

```
// 函数后直接输入 ), 说明输入的是弧度 (兼容错误输入)
currentExpr += ")";
} else if (needCompleteAngleFormula) {
    // 补全角度转换公式: ) * Math.PI / 180
    currentExpr += ") * Math.PI / 180)";
} else {
    currentExpr += ")";
}
```

重新运行实验代码，我们观察下现象：

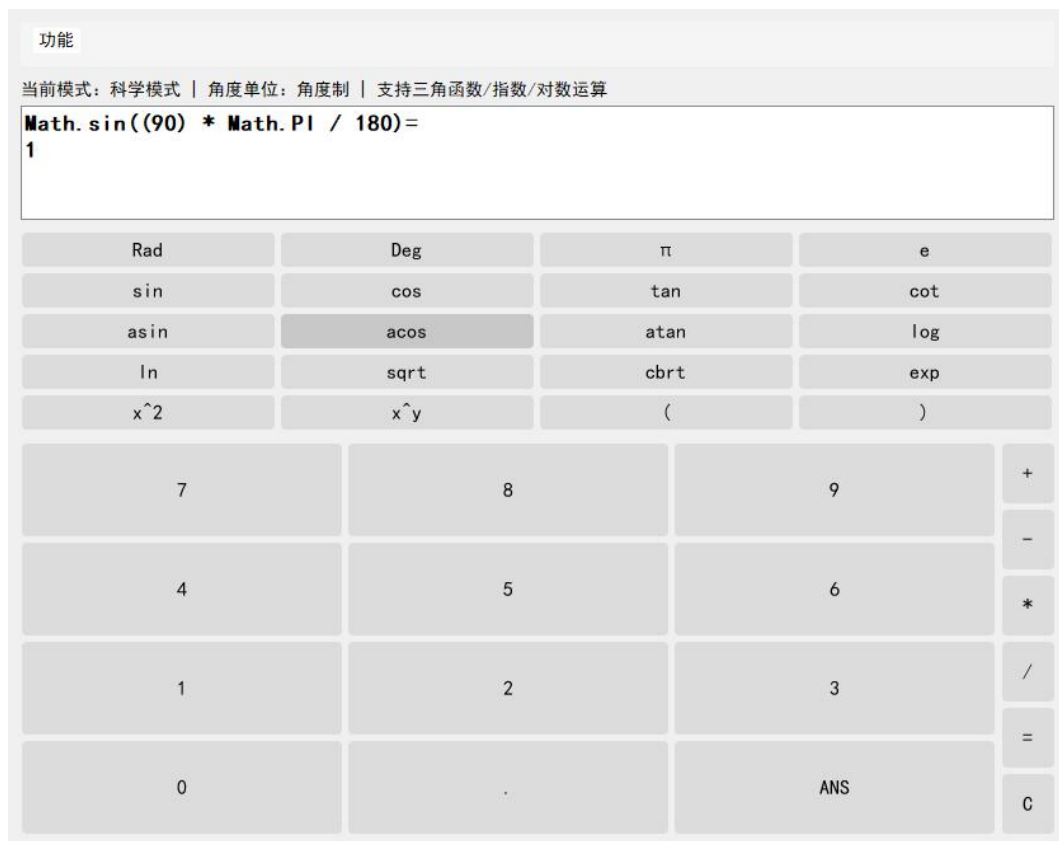


图 20 自动补全表达式

我们的逻辑构造是正确的，因此，对于角度制下的反三角函数，我们也采用以上逻辑：

```
if ("科学模式".equals(currentMode) && !isRadian && isInverseTrig) {
    double angleResult = result * 180 / Math.PI; // 弧度 → 角度
    resultStr = formatResult(String.valueOf(angleResult));
    // 显示时标注单位 (角度)
    resultStr += "°";
} else {
    resultStr = formatResult(String.valueOf(result));
}
```

对计算的结果转化后再输出，我们同样地来验证一下逻辑：

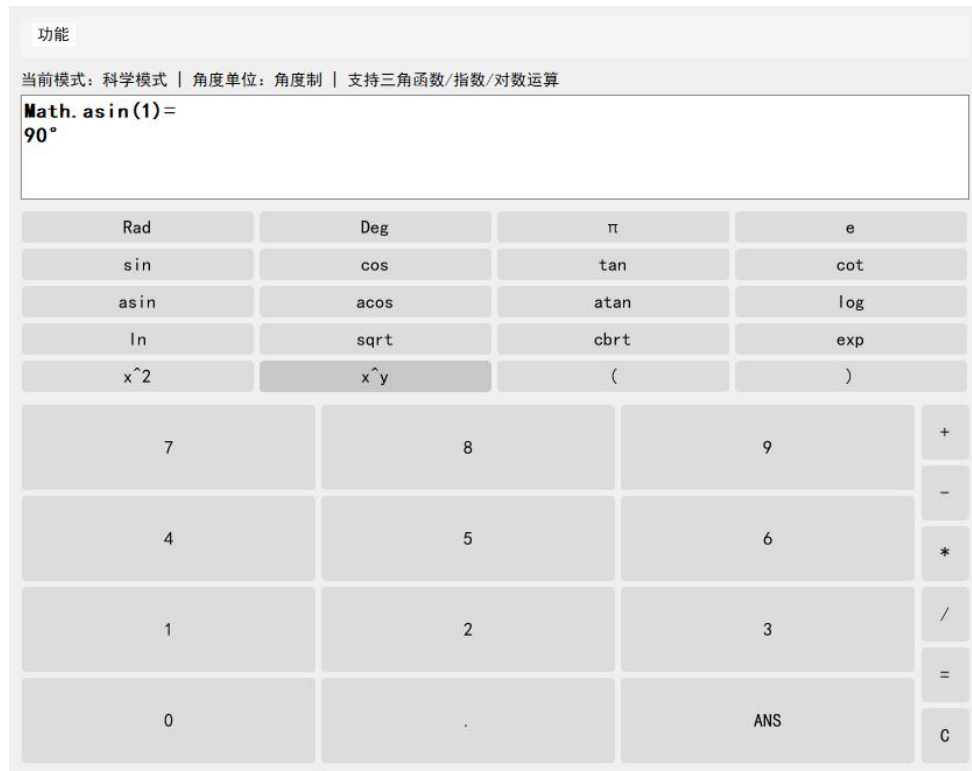


图 21 角度制下的反三角函数计算结果转化

受此启发，我们同时也添加了很多的自动补全逻辑，以优化我们使用时的体验：

```
// 检测函数后直接跟数字（如 sin30 → sin(30)）
boolean isFunctionFollowedByNumber = btnText.matches("[0-9.]")
&& !currentExpr.isEmpty() && (
    currentExpr.endsWith("sin") || currentExpr.endsWith("cos") ||
currentExpr.endsWith("tan") ||
    currentExpr.endsWith("cot") || currentExpr.endsWith("asin") ||
currentExpr.endsWith("acos") ||
    currentExpr.endsWith("atan") || currentExpr.endsWith("log") ||
currentExpr.endsWith("ln") ||
    currentExpr.endsWith("sqrt") || currentExpr.endsWith("cbrt") ||
currentExpr.endsWith("exp")
);

if (isFunctionFollowedByNumber) {
    // 函数后直接输入数字，自动补全左括号
    currentExpr += "(" + btnText;
    displayArea.setText(currentExpr);
    return;
}
```

效果如下：



图 22 自动补全右括号

2. 程序员模式的开发

程序员模式主要是实现一系列的位运算，而这也可以通过 JS 引擎的 eval 方法计算表达式的结果，因此我们可以直接将表达式输入进去，将结果得到，但这里我们的重点在于如何对不同进制下的数据进行异常处理，这里我们的思路是利用 string 类下一个方法与正则表达式进行正则匹配，如果匹配上了，则进行报错处理。

```
switch (currentRadix) {
    case "BIN":
        isValid = btnText.matches("[01]");
        break;
    case "OCT":
        isValid = btnText.matches("[0-7]");
        break;
    case "DEC":
        isValid = btnText.matches("[0-9.]"); // 允许小数点输入
        break;
    case "HEX":
        isValid = btnText.matches("[0-9A-Fa-f]");
        break;
}
```

我们来测试一下我们的代码的正确性：



图 23 非法数据输入检测

其他的部分，同基本计算器的开发，这里我们也不加赘述，最后验证整体代码的功能性：



图 24 程序员模式计算验证

可以发现，我们的功能是正确的，那我们来到了最后目标功能的开发——函数

绘图了。

五、函数绘图功能的实现

(1) 实验源代码（最终版）

```
// 包声明：计算器主类所在包
package cn;

// 导入 Swing 相关类（图形界面组件）
import javax.swing.*;
// 导入 AWT 相关类（布局、事件、绘图）
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.geom.Line2D;
// 导入 IO 相关类（文件读写，历史记录持久化）
import java.io.*;
// 导入日期格式化类（历史记录时间戳）
import java.text.SimpleDateFormat;
import java.util.Date;
// 导入集合类（存储函数数据、历史记录）
import java.util.ArrayList;
import java.util.List;
// 导入 JS 脚本引擎类（表达式计算核心）
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;
// 导入正则表达式类（输入验证、表达式处理）
import java.util.regex.Pattern;
/**
 * 版本 4：终极计算器
 * 功能清单：
 * 1. 标准模式：基础算术、记忆功能、百分比、倒数、平方根等
 * 2. 科学模式：三角函数（角度/弧度切换）、指数、对数、开方、幂运算等
 * 3. 程序员模式：进制转换（二/八/十/十六进制）、32 位位运算（JS 引擎）
 * 4. 函数绘图模式：多函数多颜色绘制、坐标范围自定义、图例显示、表达式解析
 * 5. 历史记录：本地持久化、查看、清空、双击复用（仅存储计算记录）
 */
// 计算器主类：继承 JFrame（顶层窗口），实现 ActionListener（按钮事件监听）
public class Calculate_v5 extends JFrame implements ActionListener {
    // ===== 核心成员变量 =====
    // 卡片布局管理器：用于切换不同模式面板（标准/科学/程序员/绘图）
    private CardLayout modeCardLayout;
```

```

// 卡片面板：承载所有模式面板的容器
private JPanel modeCardPanel;
// 主面板：整个窗口的根面板
private JPanel mainPanel;
// 北部面板：包含顶部控制栏和显示区域
private JPanel northPanel;
// 顶部控制面板：包含菜单栏
private JPanel topControlPanel;
// 显示面板：计算模式（非绘图）的表达式/结果显示容器
private JPanel displayPanel;
// 显示区域：文本域，用于展示表达式、结果、进制信息等
private JTextArea displayArea;
// 基础计算相关
// 当前输入的表达式字符串
private String currentExpr = "";
// 最大数字位数限制（16 位）
private final int MAX_DIGIT = 16;
// JS 脚本引擎：用于解析和计算数学表达式
private ScriptEngine jsEngine;
// 存储上一次计算结果（ANS 功能）
private String ans = "";
// 当前激活的模式（默认：标准模式）
private String currentMode = "标准模式";
// 三角函数角度单位标记（true=弧度制，false=角度制）
private boolean isRadian = true;
// 程序员模式核心状态
// 当前进制（默认：十进制 DEC，可选 BIN/OCT/HEX）
private String currentRadix = "DEC";
// 历史记录相关
// 历史记录数据模型：存储历史记录条目
private DefaultListModel<String> historyModel;
// 历史记录列表组件：展示历史记录
private JList<String> historyList;
// 日期格式化器：生成历史记录的时间戳
private SimpleDateFormat dateFormat;
// 历史记录文件路径：本地持久化存储文件
private static final String HISTORY_FILE = "calc_history.txt";
// 函数绘图模式专属变量（新增）
// 函数绘图主面板：承载输入区和绘图画布
private JPanel funcPlotPanel;
// 函数输入+参数面板：包含表达式输入、坐标范围设置、颜色选择
private JPanel funcInputPanel;
// 函数表达式按键面板：快捷输入函数符号（如 sin、x^2 等）
private JPanel funcKeyPanel;
    
```

```

// 绘图画布面板：核心绘图区域（重写 paintComponent 实现绘图）
private JPanel funcCanvasPanel;
// 函数表达式显示框：输入待绘制的函数表达式
private JTextField funcExprField;
// x 最小值输入框：自定义 x 轴左边界
private JTextField xMinField;
// x 最大值输入框：自定义 x 轴右边界
private JTextField xMaxField;
// y 最小值输入框：自定义 y 轴下边界
private JTextField yMinField;
// y 最大值输入框：自定义 y 轴上边界
private JTextField yMaxField;
// 添加函数到画布按钮：触发函数绘制
private JButton addFuncBtn;
// 清空画布按钮：清除所有已绘制函数和输入
private JButton clearFuncBtn;
// 颜色选择下拉框：选择函数曲线颜色
private JComboBox<String> colorCombo;
// 存储已添加的函数数据（表达式+颜色）
private List<FuncData> funcList;
// 坐标轴边距：画布边缘与坐标轴的距离（避免刻度/标签被截断）
private final int AXIS_MARGIN = 50;
// 默认 x 轴最小值（-10）
private double xMin = -10;
// 默认 x 轴最大值（10）
private double xMax = 10;
// 默认 y 轴最小值（-10）
private double yMin = -10;
// 默认 y 轴最大值（10）
private double yMax = 10;
// ===== 样式常量 =====
// 中文字体：用于标签、菜单等文本（黑体 14 号）
private static final Font CHINESE_FONT = new Font("黑体", Font.PLAIN, 14);
// 按钮字体：所有功能按钮文本（黑体 16 号）
private static final Font BUTTON_FONT = new Font("黑体", Font.PLAIN, 16);
// 显示区域字体：表达式/结果显示（黑体加粗 18 号）
private static final Font DISPLAY_FONT = new Font("黑体", Font.BOLD, 18);
// 列表字体：历史记录列表文本（黑体 13 号）
private static final Font LIST_FONT = new Font("黑体", Font.PLAIN, 13);
// 按钮默认背景色（浅灰色）
private static final Color BUTTON_BACKGROUND = new Color(220, 220, 220);
// 按钮 hover 背景色（较浅灰色）
private static final Color BUTTON_HOVER_BACKGROUND = new Color(200, 200, 200);
// 按钮按下背景色（深灰色）

```

```

private static final Color BUTTON_PRESS_BACKGROUND = new Color(180, 180, 180);
// 按钮禁用背景色（浅灰色，区分可用状态）
private static final Color BUTTON_DISABLED_BACKGROUND = new Color(200, 200,
200);
// 按钮文本颜色（黑色）
private static final Color BUTTON_FOREGROUND = Color.BLACK;
// 控制面板背景色（浅灰白色）
private static final Color CONTROL_PANEL_BACKGROUND = new Color(245, 245, 245);
// 菜单背景色（与控制面板一致）
private static final Color MENU_BACKGROUND = CONTROL_PANEL_BACKGROUND;
// 菜单文本颜色（黑色）
private static final Color MENU_FOREGROUND = Color.BLACK;
// 菜单选中背景色（灰色）
private static final Color MENU_SELECTED_BACKGROUND = new Color(200, 200, 200);
// 菜单选中文本颜色（黑色）
private static final Color MENU_SELECTED_FOREGROUND = Color.BLACK;
// ===== 函数数据实体类（新增） =====
// 函数数据实体类：封装单个函数的表达式和曲线颜色
class FuncData {
    // 函数表达式（如 "x^2"、"sin(x)"）
    String expr;
    // 函数曲线颜色
    Color color;
    // 构造方法：初始化表达式和颜色
    FuncData(String expr, Color color) {
        this.expr = expr;
        this.color = color;
    }
}
// ===== 自定义圆角按钮 =====
// 自定义圆角按钮类：继承 JButton，重写样式和绘制逻辑
class RoundButton extends JButton {
    // 圆角半径（8 像素）
    private int radius = 8;
    // 构造方法：接收按钮文本，初始化样式
    public RoundButton(String text) {
        super(text);
        initStyle();
    }
    // 初始化按钮样式：设置字体、颜色、边框等
    private void initStyle() {
        setForeground(BUTTON_FOREGROUND);
        setBackground(BUTTON_BACKGROUND);
        setFont(BUTTON_FONT);
    }
}
    
```

```

setBorderPainted(false); // 隐藏默认边框
setFocusPainted(true); // 保留焦点框（便于键盘操作）
setContentAreaFilled(false); // 禁用默认背景填充（自定义绘制）
// 添加鼠标事件监听器：实现 hover/press 状态切换
addMouseListener(new MouseAdapter() {
    // 鼠标进入按钮：切换 hover 背景色
    @Override
    public void mouseEntered(MouseEvent e) {
        if (isEnabled()) {
            setBackground(BUTTON_HOVER_BACKGROUND);
        }
    }
    // 鼠标离开按钮：恢复默认背景色
    @Override
    public void mouseExited(MouseEvent e) {
        if (isEnabled()) {
            setBackground(BUTTON_BACKGROUND);
        }
    }
    // 鼠标按下按钮：切换 press 背景色
    @Override
    public void mousePressed(MouseEvent e) {
        if (isEnabled()) {
            setBackground(BUTTON_PRESS_BACKGROUND);
        }
    }
    // 鼠标释放按钮：恢复 hover 背景色
    @Override
    public void mouseReleased(MouseEvent e) {
        if (isEnabled()) {
            setBackground(BUTTON_HOVER_BACKGROUND);
        }
    }
});
}
// 重写绘制组件方法：实现圆角背景和居中文本
@Override
protected void paintComponent(Graphics g) {
    // 创建 2D 绘图对象（支持抗锯齿、圆角等高级功能）
    Graphics2D g2d = (Graphics2D) g.create();
    // 开启抗锯齿（使圆角和文本更平滑）
    g2d.setRenderingHint(
        RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON
    );
}

```

```

        );
        // 设置背景色（根据启用状态切换）
        g2d.setColor(isEnabled() ? getBackground() :
BUTTON_DISABLED_BACKGROUND);
        // 绘制圆角矩形背景
        g2d.fillRoundRect(0, 0, getWidth(), getHeight(), radius, radius);
        // 设置文本颜色
        g2d.setColor(getForeground());
        // 计算文本居中位置
        FontMetrics fm = g2d.getFontMetrics();
        int textX = (getWidth() - fm.stringWidth(getText())) / 2; // 水平居
中
        int textY = (getHeight() + fm.getAscent()) / 2 - fm.getDescent(); //
垂直居中

        // 绘制按钮文本
        g2d.drawString(getText(), textX, textY);
        // 释放绘图资源
        g2d.dispose();
    }
}

// ===== 构造方法（初始化所有功能） =====
// 构造方法：初始化窗口、组件、数据模型和监听器
public Calculate_v5() {
    // 设置窗口标题
    setTitle("终极计算器（标准/科学/程序员/函数绘图 + 历史记录）");
    // 设置窗口大小（适配绘图功能，扩大至 950x750）
    setSize(950, 750);
    // 设置窗口关闭行为（退出程序）
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    // 设置窗口居中显示
    setLocationRelativeTo(null);
    // 允许窗口大小调整
    setResizable(true);
    // 初始化核心组件
    // 创建 JS 脚本引擎（用于表达式计算）
    jsEngine = new ScriptEngineManager().getEngineByName("JavaScript");
    // 初始化历史记录数据模型
    historyModel = new DefaultListModel<>();
    // 绑定历史记录列表与数据模型
    historyList = new JList<>(historyModel);
    historyList.setFont(LIST_FONT);
    // 设置历史记录列表选择模式（单选）
    historyList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    // 初始化日期格式化器（时间戳格式：时:分:秒）

```

```

        dateFormat = new SimpleDateFormat("HH:mm:ss");
        // 初始化函数绘图相关（新增）
        // 创建函数数据列表（存储已添加的函数）
        funcList = new ArrayList<>();
        // 加载历史记录并初始化界面
        loadHistory(); // 从本地文件加载历史记录
        initHistoryListDoubleClick(); // 初始化历史记录双击复用功能
        initLayout(); // 初始化整体布局
        initComponents(); // 初始化所有 UI 组件
        initListeners(); // 初始化所有事件监听器
        // 显示窗口
        setVisible(true);
    }
    // ===== 布局初始化（新增函数绘图面板布局）
    =====
    // 初始化整体布局：设置各面板的层级关系和布局管理器
    private void initLayout() {
        // 主面板：使用边界布局（上下左右中），设置内边距
        mainPanel = new JPanel(new BorderLayout(10, 10));
        mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        add(mainPanel); // 将主面板添加到窗口
        // 北部面板：包含顶部控制栏和显示区域（边界布局）
        northPanel = new JPanel(new BorderLayout(5, 5));
        // 顶部控制面板：流式布局（左对齐），设置背景色和固定高度
        topControlPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
        topControlPanel.setBackground(CONTROL_PANEL_BACKGROUND);
        topControlPanel.setPreferredSize(new Dimension(930, 35));
        // 显示面板：计算模式的表达式/结果显示容器（边界布局）
        displayPanel = new JPanel(new BorderLayout(5, 5));
        // 卡片布局管理器：用于切换 4 种模式面板
        modeCardLayout = new CardLayout();
        // 卡片面板：承载所有模式面板（标准/科学/程序员/绘图）
        modeCardPanel = new JPanel(modeCardLayout);
        // 组装北部面板：顶部控制栏（北）+ 显示区域（南）
        northPanel.add(topControlPanel, BorderLayout.NORTH);
        northPanel.add(displayPanel, BorderLayout.SOUTH);
        // 组装主面板：北部面板（北）+ 模式卡片面板（中）
        mainPanel.add(northPanel, BorderLayout.NORTH);
        mainPanel.add(modeCardPanel, BorderLayout.CENTER);
    }
    // ===== 组件初始化（核心：新增函数绘图组件）
    =====
    // 初始化所有 UI 组件：菜单、显示区域、各模式面板
    private void initComponents() {

```

```

// 1. 顶部菜单（新增"函数绘图模式"选项）
// 创建菜单条
JMenuBar menuBar = new JMenuBar();
menuBar.setBackground(MENU_BACKGROUND);
menuBar.setBorderPainted(false); // 隐藏菜单条边框
// "功能"菜单：包含模式切换、历史记录操作
JMenu funcMenu = new JMenu("功能");
funcMenu.setFont(CHINESE_FONT);
funcMenu.setBackground(MENU_BACKGROUND);
funcMenu.setForeground(MENU_FOREGROUND);
// "模式切换"子菜单：包含 4 种模式选项
JMenu modeMenu = new JMenu("模式切换");
modeMenu.setFont(CHINESE_FONT);
// 模式选项数组（新增"函数绘图模式"）
String[] modes = {"标准模式", "科学模式", "程序员模式", "函数绘图模式"};
// 为每个模式创建菜单选项并添加到子菜单
for (String mode : modes) {
    JMenuItem menuItem = new JMenuItem(mode);
    menuItem.setFont(CHINESE_FONT);
    modeMenu.add(menuItem);
}
// 历史记录相关菜单选项
JMenuItem historyItem = new JMenuItem("查看历史记录");
historyItem.setFont(CHINESE_FONT);
JMenuItem clearHistoryItem = new JMenuItem("清空历史记录");
clearHistoryItem.setFont(CHINESE_FONT);
// 设置菜单选项选中状态样式
UIManager.put("MenuItem.selectionBackground",
MENU_SELECTED_BACKGROUND);
UIManager.put("MenuItem.selectionForeground",
MENU_SELECTED_FOREGROUND);
// 组装"功能"菜单：模式切换子菜单 + 分隔线 + 历史记录操作
funcMenu.add(modeMenu);
funcMenu.addSeparator();
funcMenu.add(historyItem);
funcMenu.add(clearHistoryItem);
// 将"功能"菜单添加到菜单条
menuBar.add(funcMenu);
// 将菜单条添加到顶部控制面板
topControlPanel.add(menuBar);
// 2. 计算模式的显示区域（标准/科学/程序员模式使用）
// 提示标签：显示当前模式和操作提示
JLabel tipLabel = new JLabel(
    "当前模式：标准模式 | 提示：通过\"功能\"菜单切换模式和查看历史记录"

```

```

    );
    tipLabel.setFont(CHINESE_FONT);
    displayPanel.add(tipLabel, BorderLayout.NORTH); // 添加到显示面板北部
    // 显示文本域：展示表达式、结果、进制信息等
    displayArea = new JTextArea(4, 50);
    displayArea.setFont(DISPLAY_FONT);
    displayArea.setEditable(false); // 禁止手动编辑
    displayArea.setLineWrap(true); // 自动换行
    // 添加滚动条（当内容超出时）
    displayPanel.add(new JScrollPane(displayArea), BorderLayout.CENTER);
    // 3. 添加 4 种模式的面板（新增函数绘图模式面板）
    modeCardPanel.add(new StandardModePanel(), "标准模式"); // 标准模式面板
    modeCardPanel.add(new ScientificModePanel(), "科学模式"); // 科学模式面板
    modeCardPanel.add(new ProgrammerModePanel(), "程序员模式"); // 程序员模式
    // 新增：函数绘图模式面板
    modeCardPanel.add(createFuncPlotPanel(), "函数绘图模式"); // 新增：函数绘图模式面板
}
// ===== 模式面板实现（保留原有+新增绘图面板）
=====

// 程序员模式面板：包含进制切换、位运算、数字键盘
class ProgrammerModePanel extends JPanel {
    // 构造方法：初始化程序员模式面板布局和组件
    public ProgrammerModePanel() {
        setLayout(new BorderLayout(10, 10)); // 边界布局
        // 进制切换面板：4 个按钮（二/八/十/十六进制），网格布局（1 行 4 列）
        JPanel radixPanel = new JPanel(new GridLayout(1, 4, 5, 5));
        String[] radixTexts = {"二进制(BIN)", "八进制(OCT)", "十进制(DEC)", "十六进制(HEX)"};
        for (String text : radixTexts) {
            RoundButton btn = new RoundButton(text);
            btn.addActionListener(Calculate_v5.this); // 绑定事件监听
            radixPanel.add(btn);
        }
        // 位运算面板：8 个按钮（&/|/^/~等），网格布局（2 行 4 列）
        JPanel funcPanel = new JPanel(new GridLayout(2, 4, 5, 5));
        String[] funcTexts = {"&", "|", "^", "~", ">>", "<<", ">>>", "!"};
        for (String text : funcTexts) {
            RoundButton btn = new RoundButton(text);
            btn.addActionListener(Calculate_v5.this); // 绑定事件监听
            funcPanel.add(btn);
        }
        // 数字键盘面板：包含数字、字母（A-F）、运算符、功能键，网格布局（6 行 4 列）
        JPanel keypadPanel = new JPanel(new GridLayout(6, 4, 5, 5));
    }
}

```

```

String[] keyTexts = {
    "7", "8", "9", "A",
    "4", "5", "6", "B",
    "1", "2", "3", "C",
    "0", ".", "+", "D",
    "-", "*", "/", "E",
    "=", "C", "DEL", "F"
};
for (String text : keyTexts) {
    RoundButton btn = new RoundButton(text);
    btn.addActionListener(Calculate_v5.this); // 绑定事件监听
    keypadPanel.add(btn);
}
// 顶部面板：包含进制切换和位运算面板（边界布局）
JPanel topPanel = new JPanel(new BorderLayout(10, 10));
topPanel.add(radixPanel, BorderLayout.NORTH);
topPanel.add(funcPanel, BorderLayout.CENTER);
// 组装程序员模式面板：顶部面板（北）+ 数字键盘（中）
add(topPanel, BorderLayout.NORTH);
add(keypadPanel, BorderLayout.CENTER);
}
}
// 标准模式面板：包含基础算术、记忆功能、百分比等
class StandardModePanel extends JPanel {
    // 构造方法：初始化标准模式面板布局和组件
    public StandardModePanel() {
        // 网格布局（6行5列），设置组件间距
        setLayout(new GridLayout(6, 5, 5, 5));
        // 标准模式按钮文本数组
        String[] standardBtns = {
            "MC", "MS", "M+", "M-", "DEL", // 记忆功能+删除
            "7", "8", "9", "/", "%", // 数字+除法+百分比
            "4", "5", "6", "*", "1/x", // 数字+乘法+倒数
            "1", "2", "3", "-", "√", // 数字+减法+平方根
            "0", ".", "ANS", "+", "=", // 数字+小数点+上结果+加法+等于
            "C", "CE", "+/-", "(", ")" // 清除+正负切换+括号
        };
        // 遍历创建按钮并添加到面板
        for (String text : standardBtns) {
            RoundButton btn = new RoundButton(text);
            btn.addActionListener(Calculate_v5.this); // 绑定事件监听
            add(btn);
        }
    }
}

```

```

    }
    // 科学模式面板：包含三角函数、指数、对数、幂运算等
    class ScientificModePanel extends JPanel {
        // 构造方法：初始化科学模式面板布局和组件
        public ScientificModePanel() {
            setLayout(new BorderLayout(10, 10)); // 边界布局
            // 顶部功能面板：三角函数、指数、对数等，网格布局（5 行 4 列）
            JPanel topFunctionsPanel = new JPanel(new GridLayout(5, 4, 5, 5));
            String[] topFunctionButtons = {
                "Rad", "Deg", "π", "e", // 角度单位切换+常数π/e
                "sin", "cos", "tan", "cot", // 三角函数
                "asin", "acos", "atan", "log", // 反三角函数+常用对数
                "ln", "sqrt", "cbrt", "exp", // 自然对数+平方根+立方根+指数
                "x^2", "x^y", "(", ")" // 平方+幂运算+括号
            };
            for (String text : topFunctionButtons) {
                RoundButton btn = new RoundButton(text);
                btn.addActionListener(Calculate_v5.this); // 绑定事件监听
                topFunctionsPanel.add(btn);
            }
            // 底部键盘面板：包含数字、运算符，边界布局
            JPanel bottomKeypadPanel = new JPanel(new BorderLayout(5, 5));
            // 数字面板：网格布局（4 行 3 列）
            JPanel numberPanel = new JPanel(new GridLayout(4, 3, 5, 5));
            String[] numberButtons = {"7", "8", "9", "4", "5", "6", "1", "2", "3",
"0", ".", "ANS"};
            for (String text : numberButtons) {
                RoundButton btn = new RoundButton(text);
                btn.setPreferredSize(new Dimension(60, 60)); // 设置按钮大小
                btn.addActionListener(Calculate_v5.this); // 绑定事件监听
                numberPanel.add(btn);
            }
            // 运算符面板：网格布局（6 行 1 列）
            JPanel operatorPanel = new JPanel(new GridLayout(6, 1, 5, 5));
            String[] operatorButtons = {"+", "-", "*", "/", "=", "C"};
            for (String text : operatorButtons) {
                RoundButton btn = new RoundButton(text);
                btn.addActionListener(Calculate_v5.this); // 绑定事件监听
                operatorPanel.add(btn);
            }
            // 组装底部键盘面板：数字面板（中）+ 运算符面板（东）
            bottomKeypadPanel.add(numberPanel, BorderLayout.CENTER);
            bottomKeypadPanel.add(operatorPanel, BorderLayout.EAST);
        }
    }

```

```

        // 组装科学模式面板：顶部功能面板（北）+ 底部键盘（中）
        add(topFunctionsPanel, BorderLayout.NORTH);
        add(bottomKeypadPanel, BorderLayout.CENTER);
    }
}

// ===== 核心新增：函数绘图模式面板 =====
// 创建函数绘图模式面板：包含输入区、绘图画布
private JPanel createFuncPlotPanel() {
    // 函数绘图主面板：边界布局
    funcPlotPanel = new JPanel(new BorderLayout(10, 10));
    // 1. 函数输入+参数面板（顶部）：边界布局
    funcInputPanel = new JPanel(new BorderLayout(8, 8));
    // 1.1 参数输入区域（x/y 范围、颜色选择、功能按钮）：网格布局（2 行 6 列）
    JPanel paramPanel = new JPanel(new GridLayout(2, 6, 8, 8));
    // 第一行：函数表达式、x 范围
    JLabel exprLabel = new JLabel("函数表达式(y=f(x)):");
    exprLabel.setFont(CHINESE_FONT);
    paramPanel.add(exprLabel);
    // 函数表达式输入框
    funcExprField = new JTextField("");
    funcExprField.setFont(CHINESE_FONT);
    paramPanel.add(funcExprField);
    // x 最小值标签和输入框
    JLabel xminLabel = new JLabel("x 最小值:");
    xminLabel.setFont(CHINESE_FONT);
    paramPanel.add(xminLabel);
    xminField = new JTextField(String.valueOf(xMin));
    xminField.setFont(CHINESE_FONT);
    paramPanel.add(xminField);
    // x 最大值标签和输入框
    JLabel xmaxLabel = new JLabel("x 最大值:");
    xmaxLabel.setFont(CHINESE_FONT);
    paramPanel.add(xmaxLabel);
    xmaxField = new JTextField(String.valueOf(xMax));
    xmaxField.setFont(CHINESE_FONT);
    paramPanel.add(xmaxField);
    // 第二行：y 范围、颜色选择、功能按钮
    // y 最小值标签和输入框
    JLabel yminLabel = new JLabel("y 最小值:");
    yminLabel.setFont(CHINESE_FONT);
    paramPanel.add(yminLabel);
    yminField = new JTextField(String.valueOf(yMin));
    yminField.setFont(CHINESE_FONT);
    paramPanel.add(yminField);
}

```

```

// y 最大值标签和输入框
JLabel ymaxLabel = new JLabel("y 最大值:");
ymaxLabel.setFont(CHINESE_FONT);
paramPanel.add(ymaxLabel);
yMaxField = new JTextField(String.valueOf(yMax));
yMaxField.setFont(CHINESE_FONT);
paramPanel.add(yMaxField);
// 颜色选择标签和下拉框
JLabel colorLabel = new JLabel("曲线颜色:");
colorLabel.setFont(CHINESE_FONT);
paramPanel.add(colorLabel);
String[] colorNames = {"红色", "绿色", "蓝色", "黑色", "橙色", "紫色"};
colorCombo = new JComboBox<>(colorNames);
colorCombo.setFont(CHINESE_FONT);
paramPanel.add(colorCombo);
// 功能按钮（添加到画布、清空画布）
addFuncBtn = new RoundButton("添加到画布");
addFuncBtn.setFont(CHINESE_FONT);
paramPanel.add(addFuncBtn);
clearFuncBtn = new RoundButton("清空画布");
clearFuncBtn.setFont(CHINESE_FONT);
paramPanel.add(clearFuncBtn);
// 1.2 函数表达式按键面板（输入函数用）：网格布局（4 行 6 列）
funcKeyPanel = new JPanel(new GridLayout(4, 6, 4, 4));
String[] funcKeys = {
    "x", "^", "√", "sin(x)", "cos(x)", "tan(x)",
    "π", "e", "+", "/", "(", ")",
    ".", "清除", "确认"
};
};
// 遍历创建按钮并绑定事件
for (String key : funcKeys) {
    RoundButton btn = new RoundButton(key);
    btn.setFont(CHINESE_FONT);
    // 函数按键的点击事件（单独处理，避免与其他模式冲突）
    btn.addActionListener(e -> {
        String text = ((JButton) e.getSource()).getText();
        if ("清除".equals(text)) {
            funcExprField.setText(""); // 清空输入框
        } else if ("确认".equals(text)) {
            addFunctionToCanvas(); // 确认后直接添加到画布
        } else if ("√".equals(text)) {
            funcExprField.setText(funcExprField.getText() +
                "Math.sqrt("); // 平方根→JS 语法
        } else if ("π".equals(text)) {
    
```

```

        funcExprField.setText(funcExprField.getText() + Math.PI); //
插入π常量
    } else if ("e".equals(text)) {
        funcExprField.setText(funcExprField.getText() + Math.E); //
插入 e 常量
    } else if ("sin(x)".equals(text)) {
        funcExprField.setText(funcExprField.getText() + "sin("); //
正弦函数→JS 语法
    } else if ("cos(x)".equals(text)) {
        funcExprField.setText(funcExprField.getText() + "cos("); //
余弦函数→JS 语法
    } else if ("tan(x)".equals(text)) {
        funcExprField.setText(funcExprField.getText() + "tan("); //
正切函数→JS 语法
    } else {
        funcExprField.setText(funcExprField.getText() + text); // 其
他字符直接拼接
    }
});
funcKeyPanel.add(btn);
}
// 组装输入面板：参数面板（北）+ 函数按键面板（中）
funcInputPanel.add(paramPanel, BorderLayout.NORTH);
funcInputPanel.add(funcKeyPanel, BorderLayout.CENTER);
// 2. 绘图画布（核心）：重写 paintComponent 实现绘图
funcCanvasPanel = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g); // 调用父类绘制（确保背景等正常显示）
        Graphics2D g2d = (Graphics2D) g; // 转换为 2D 绘图对象
        // 开启抗锯齿（使线条、文本更平滑）
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        int canvasWidth = getWidth(); // 画布宽度
        int canvasHeight = getHeight(); // 画布高度
        // 绘制白色画布背景（避免与网格/坐标轴混淆）
        g2d.setColor(Color.WHITE);
        g2d.fillRect(AXIS_MARGIN, AXIS_MARGIN, canvasWidth - 2 *
AXIS_MARGIN, canvasHeight - 2 * AXIS_MARGIN);
        // 绘制网格线
        drawGrid(g2d, canvasWidth, canvasHeight);
        // 绘制坐标轴（含箭头、刻度、标签）
        drawAxis(g2d, canvasWidth, canvasHeight);
    }
};

```

```

        // 绘制所有已添加的函数曲线
        drawAllFunctions(g2d, canvasWidth, canvasHeight);
        // 绘制图例（右上角，显示函数表达式和颜色）
        drawLegend(g2d, canvasWidth);
    }
};
funcCanvasPanel.setBackground(Color.LIGHT_GRAY); // 画布背景色（浅灰色）
// 组装函数绘图主面板：输入面板（北）+ 绘图画布（中）
funcPlotPanel.add(funcInputPanel, BorderLayout.NORTH);
funcPlotPanel.add(funcCanvasPanel, BorderLayout.CENTER);
return funcPlotPanel;
}
// ===== 函数绘图辅助方法（新增） =====
/**
 * 绘制网格线：平行于 x 轴和 y 轴的辅助线，提升可读性
 */
private void drawGrid(Graphics2D g2d, int canvasWidth, int canvasHeight) {
    // 网格线颜色（浅灰色，不遮挡坐标轴和曲线）
    g2d.setColor(new Color(220, 220, 220));
    // x 轴网格步长：画布有效宽度（扣除边距）分为 10 等份
    int gridStepX = (canvasWidth - 2 * AXIS_MARGIN) / 10;
    // y 轴网格步长：画布有效高度（扣除边距）分为 10 等份
    int gridStepY = (canvasHeight - 2 * AXIS_MARGIN) / 10;
    // 绘制垂直网格线（平行于 y 轴）
    for (int x = AXIS_MARGIN; x <= canvasWidth - AXIS_MARGIN; x += gridStepX)
    {
        g2d.drawLine(x, AXIS_MARGIN, x, canvasHeight - AXIS_MARGIN);
    }
    // 绘制水平网格线（平行于 x 轴）
    for (int y = AXIS_MARGIN; y <= canvasHeight - AXIS_MARGIN; y += gridStepY)
    {
        g2d.drawLine(AXIS_MARGIN, y, canvasWidth - AXIS_MARGIN, y);
    }
}
/**
 * 绘制坐标轴（x 轴+y 轴）：包含箭头、刻度、刻度值、轴标签
 */
private void drawAxis(Graphics2D g2d, int canvasWidth, int canvasHeight) {
    // 坐标轴颜色（黑色，突出显示）
    g2d.setColor(Color.BLACK);
    // 坐标轴线宽（2 像素，加粗）
    g2d.setStroke(new BasicStroke(2));
    // 计算坐标轴位置（根据当前 x/y 范围动态调整，确保原点在合理位置）
    int xAxisY = yToPixel(0, canvasHeight); // x 轴的 y 坐标（屏幕坐标系）

```

```

int yAxisX = xToPixel(0, canvasWidth); // y 轴的 x 坐标（屏幕坐标系）
// 绘制 x 轴（水平方向）
g2d.drawLine(Axis_Margin, xAxisY, canvasWidth - Axis_Margin, xAxisY);
// 绘制 y 轴（垂直方向）
g2d.drawLine(yAxisX, Axis_Margin, yAxisX, canvasHeight - Axis_Margin);
// 绘制 x 轴箭头（向右）：两个线段组成箭头形状
drawArrow(g2d, canvasWidth - Axis_Margin, xAxisY, canvasWidth -
Axis_Margin - 10, xAxisY - 5);
drawArrow(g2d, canvasWidth - Axis_Margin, xAxisY, canvasWidth -
Axis_Margin - 10, xAxisY + 5);
// 绘制 y 轴箭头（向上）：两个线段组成箭头形状
drawArrow(g2d, yAxisX, Axis_Margin, yAxisX - 5, Axis_Margin + 10);
drawArrow(g2d, yAxisX, Axis_Margin, yAxisX + 5, Axis_Margin + 10);
// 绘制刻度和刻度值：恢复默认线宽（1 像素）
g2d.setStroke(new BasicStroke(1));
// 刻度值字体（Arial 10 号）
g2d.setFont(new Font("Arial", Font.PLAIN, 10));
// x 轴刻度步长：x 轴范围分为 10 等份
double xTickStep = (xMax - xMin) / 10;
// y 轴刻度步长：y 轴范围分为 10 等份
double yTickStep = (yMax - yMin) / 10;
// 绘制 x 轴刻度和刻度值
for (int i = 0; i <= 10; i++) {
    double x = xMin + i * xTickStep; // 当前刻度的 x 值（数学坐标系）
    int pixelX = xToPixel(x, canvasWidth); // 转换为屏幕像素坐标
    // 绘制刻度线（垂直方向，长度 10 像素）
    g2d.drawLine(pixelX, xAxisY - 5, pixelX, xAxisY + 5);
    // 跳过原点（避免与 y 轴标签重叠），绘制刻度值
    if (Math.abs(x) >= 0.01) {
        g2d.drawString(String.format("%.1f", x), pixelX - 15, xAxisY + 20);
    }
}
// 绘制 y 轴刻度和刻度值
for (int i = 0; i <= 10; i++) {
    double y = yMin + i * yTickStep; // 当前刻度的 y 值（数学坐标系）
    int pixelY = yToPixel(y, canvasHeight); // 转换为屏幕像素坐标
    // 绘制刻度线（水平方向，长度 10 像素）
    g2d.drawLine(yAxisX - 5, pixelY, yAxisX + 5, pixelY);
    // 跳过原点（避免与 x 轴标签重叠），绘制刻度值
    if (Math.abs(y) >= 0.01) {
        g2d.drawString(String.format("%.1f", y), yAxisX - 30, pixelY + 5);
    }
}
}
    
```

```

        // 绘制坐标轴标签 (x/y 轴名称)：加粗字体 (Arial 14 号)
        g2d.setFont(new Font("Arial", Font.BOLD, 14));
        g2d.drawString("x", canvasWidth - AXIS_MARGIN + 10, xAxisY - 5); // x 轴
        g2d.drawString("y", yAxisX - 15, AXIS_MARGIN - 10); // y 轴标签
    }
    /**
     * 绘制箭头 (辅助方法)：绘制单条线段，用于坐标轴箭头
     * @param g2d 2D 绘图对象
     * @param x1 起点 x 坐标
     * @param y1 起点 y 坐标
     * @param x2 终点 x 坐标
     * @param y2 终点 y 坐标
     */
    private void drawArrow(Graphics2D g2d, int x1, int y1, int x2, int y2) {
        g2d.drawLine(x1, y1, x2, y2);
    }
    /**
     * 绘制所有已添加的函数曲线：遍历 funcList，计算每个函数的采样点并连线
     */
    private void drawAllFunctions(Graphics2D g2d, int canvasWidth, int
canvasHeight) {
        // 采样点数：确保曲线平滑 (最小 2000 点，或根据画布宽度自适应)
        int sampleCount = Math.max(2000, (canvasWidth - 2 * AXIS_MARGIN) / 2);
        // x 轴采样步长：均匀分布在 xMin 到 xMax 之间
        double xStep = (xMax - xMin) / (sampleCount - 1);
        // 记录绘制失败的函数 (用于后续提示)
        List<String> failedFuncs = new ArrayList<>();
        // 遍历每个已添加的函数
        for (FuncData func : funcList) {
            g2d.setColor(func.color); // 设置当前函数曲线颜色
            g2d.setStroke(new BasicStroke(2)); // 曲线线宽 (2 像素)
            Point prevPoint = null; // 上一个有效采样点 (用于连线)
            int validCount = 0; // 有效采样点计数 (判断函数是否可绘制)
            // 遍历所有采样点，计算并绘制
            for (int i = 0; i < sampleCount; i++) {
                double x = xMin + i * xStep; // 当前采样点的 x 值 (数学坐标系)
                double y = calculateFuncY(func.expr, x); // 计算对应的 y 值
                // 验证 y 值有效性：非 NaN、非无穷大、在 y 轴范围内
                if (!Double.isNaN(y) && !Double.isInfinite(y) && y >= yMin && y <=
yMax) {
                    validCount++;
                    // 转换为屏幕像素坐标
                    int pixelX = xToPixel(x, canvasWidth);

```

```

        int pixelY = yToPixel(y, canvasHeight);
        Point currentPoint = new Point(pixelX, pixelY);
        // 连接当前点和上一个有效点（绘制线段）
        if (prevPoint != null) {
            g2d.draw(new Line2D.Double(prevPoint.x, prevPoint.y,
currentPoint.x, currentPoint.y));
        }
        prevPoint = currentPoint; // 更新上一个有效点
    } else {
        prevPoint = null; // 无效点，断开曲线（避免绘制错误线段）
    }
}
// 无有效点：记录该函数绘制失败
if (validCount == 0) {
    failedFuncs.add("y = " + func.expr);
}
}
// 提示绘制失败的函数（通过 SwingUtilities 确保在 EDT 线程执行）
if (!failedFuncs.isEmpty()) {
    SwingUtilities.invokeLater(() -> {
        String msg = "以下函数在当前坐标范围无法绘制，请检查表达式或调整 x/y
范围：\n\n";
        msg += String.join("\n", failedFuncs);
        JOptionPane.showMessageDialog(this, msg, "绘制失败",
JOptionPane.WARNING_MESSAGE);
    });
}
}
/**
 * 计算函数在指定 x 值下的 y 值（核心：表达式解析和计算）
 * @param funcExpr 函数表达式（如 "x^2"、"sin(x)"）
 * @param x 当前 x 值（数学坐标系）
 * @return 对应的 y 值（无效时返回 NaN）
 */
private double calculateFuncY(String funcExpr, double x) {
    try {
        // 表达式转换：适配 JavaScript 语法（与科学模式保持一致）
        String expr = funcExpr
            .replaceAll("x\\^[0-9.]+)", "Math.pow(x,$1)") // x^n →
Math.pow(x,n)（如 x^2→Math.pow(x,2)）
            .replaceAll("\\^", "Math.pow") // 独立^ → Math.pow（如
2^3→Math.pow(2,3)）
            .replaceAll("\\*\\*", "Math.pow") // ** → Math.pow（兼容其他写法）
    }
}

```

```

        .replaceAll("sin\\((.*)\\)", "Math.sin($1)") // sin(...) →
Math.sin(...)
        .replaceAll("cos\\((.*)\\)", "Math.cos($1)") // cos(...) →
Math.cos(...)
        .replaceAll("tan\\((.*)\\)", "Math.tan($1)") // tan(...) →
Math.tan(...)
        .replaceAll("x", "(" + x + ")"); // 替换 x 为当前值（加括号避免运算
优先级问题）
        // 执行 JS 表达式计算
        Object result = jsEngine.eval(expr);
        // 转换为 double 类型返回（非数字类型返回 NaN）
        return result instanceof Number ? ((Number) result).doubleValue() :
Double.NaN;
    } catch (ScriptException | ClassCastException e) {
        // 表达式解析失败或类型转换失败：返回 NaN
        return Double.NaN;
    }
}

/**
 * x 坐标转换：数学坐标系 → 屏幕像素坐标系
 * @param x 数学坐标系的 x 值
 * @param canvasWidth 画布宽度
 * @return 屏幕像素 x 坐标
 */
private int xToPixel(double x, int canvasWidth) {
    // 计算 x 轴缩放比例（像素/数学单位）
    double ratioX = (double) (canvasWidth - 2 * AXIS_MARGIN) / (xMax - xMin);
    // 转换公式：边距 + (x - xMin) * 缩放比例
    return (int) (AXIS_MARGIN + (x - xMin) * ratioX);
}

/**
 * y 坐标转换：数学坐标系 → 屏幕像素坐标系（屏幕 y 轴向下，需翻转）
 * @param y 数学坐标系的 y 值
 * @param canvasHeight 画布高度
 * @return 屏幕像素 y 坐标
 */
private int yToPixel(double y, int canvasHeight) {
    // 计算 y 轴缩放比例（像素/数学单位）
    double ratioY = (double) (canvasHeight - 2 * AXIS_MARGIN) / (yMax - yMin);
    // 转换公式：画布高度 - 边距 - (y - yMin) * 缩放比例（翻转 y 轴）
    return (int) (canvasHeight - AXIS_MARGIN - (y - yMin) * ratioY);
}

/**
 * 绘制图例：在画布右上角显示每个函数的表达式和对应颜色

```

```

    */
    private void drawLegend(Graphics2D g2d, int canvasWidth) {
        g2d.setColor(Color.BLACK); // 图例文本颜色（黑色）
        g2d.setFont(CHINESE_FONT); // 图例字体（与其他文本一致）
        // 图例位置：右上角（距离右边距 AXIS_MARGIN+180 像素，上边距 AXIS_MARGIN+20
        // 像素）
        int legendX = canvasWidth - AXIS_MARGIN - 180;
        int legendY = AXIS_MARGIN + 20;
        int legendStep = 25; // 每个图例项的垂直间距（25 像素）
        // 遍历每个函数，绘制图例项
        for (int i = 0; i < funcList.size(); i++) {
            FuncData func = funcList.get(i);
            int currentY = legendY + i * legendStep; // 当前图例项的 y 坐标
            // 绘制颜色块（20x20 像素）
            g2d.setColor(func.color);
            g2d.fillRect(legendX, currentY - 10, 20, 20);
            // 绘制颜色块边框（黑色，突出显示）
            g2d.setColor(Color.BLACK);
            g2d.drawRect(legendX, currentY - 10, 20, 20);
            // 绘制函数表达式（颜色块右侧）
            g2d.drawString("y = " + func.expr, legendX + 25, currentY + 5);
        }
    }
    /**
     * 添加函数到画布：验证输入合法性，添加到 funcList 并刷新画布
     */
    private void addFunctionToCanvas() {
        // 获取函数表达式（去除首尾空格）
        String inputExpr = funcExprField.getText().trim();
        // 表达式为空：提示错误
        if (inputExpr.isEmpty()) {
            JOptionPane.showMessageDialog(this, "请输入函数表达式！", "错误",
            JOptionPane.ERROR_MESSAGE);
            return;
        }
        // 验证坐标范围输入合法性
        try {
            // 解析 x/y 范围输入值
            xMin = Double.parseDouble(xMinField.getText().trim());
            xMax = Double.parseDouble(xMaxField.getText().trim());
            yMin = Double.parseDouble(yMinField.getText().trim());
            yMax = Double.parseDouble(yMaxField.getText().trim());
            // 最小值 >= 最大值：提示错误
            if (xMin >= xMax || yMin >= yMax) {

```

```

        JOptionPane.showMessageDialog(this, "最小值必须小于最大值!", "错误",
JOptionPane.ERROR_MESSAGE);
        return;
    }
} catch (NumberFormatException e) {
    // 输入非数字: 提示错误
    JOptionPane.showMessageDialog(this, "请输入有效的数字!", "错误",
JOptionPane.ERROR_MESSAGE);
    return;
}
// 映射颜色选择: 下拉框索引 → 实际 Color 对象
Color[] colors = {Color.RED, Color.GREEN, Color.BLUE, Color.BLACK,
Color.ORANGE, Color.MAGENTA};
Color selectedColor = colors[colorCombo.getSelectedIndex()];
// 添加函数到列表, 并刷新画布 (触发 paintComponent 重绘)
funcList.add(new FuncData(inputExpr, selectedColor));
funcCanvasPanel.repaint();
}
/**
 * 清空画布: 清除所有函数数据, 重置输入框和坐标范围
 */
private void clearFunctionCanvas() {
    funcList.clear(); // 清空函数列表
    // 重置默认坐标范围
    xMin = -10;
    xMax = 10;
    yMin = -10;
    yMax = 10;
    // 重置输入框和颜色选择
    funcExprField.setText("");
    xMinField.setText(String.valueOf(xMin));
    xMaxField.setText(String.valueOf(xMax));
    yMinField.setText(String.valueOf(yMin));
    yMaxField.setText(String.valueOf(yMax));
    colorCombo.setSelectedIndex(0);
    // 刷新画布 (清除所有绘制内容)
    funcCanvasPanel.repaint();
}
// ===== 原有核心方法 (保持不变) =====
// 程序员模式核心方法: 切换当前进制 (BIN/OCT/DEC/HEX)
private void switchRadix(String radix) {
    currentRadix = radix; // 更新当前进制
    // 更新提示标签文本
    JLabel tipLabel = (JLabel) displayPanel.getComponent(0);

```

```

        tiplabel.setText(
            "当前模式：程序员模式 | 当前进制： " + currentRadix + " | 支持 32 位整数+
位运算 (JS 引擎)"
        );
        // 如果当前有计算结果，更新进制显示
        if (!currentExpr.isEmpty() && currentExpr.contains(" = ")) {
            try {
                // 提取结果字符串（分割" = "后的部分）
                String resultStr = currentExpr.split(" = ")[1].trim();
                int value = Integer.parseInt(resultStr); // 转换为 int (32 位)
                updateProgrammerDisplay(value); // 更新进制显示
            } catch (Exception e) {
                // 结果无法解析（如科学计数法）：不更新显示
            }
        }
    }
    // 程序员模式：更新进制显示（展示二/八/十/十六进制对应值）
    private void updateProgrammerDisplay(int value) {
        // 转换为各进制字符串（二进制 32 位补零，十六进制大写）
        String binStr = String.format("%32s",
Integer.toBinaryString(value)).replace(' ', '0');
        String octStr = Integer.toOctalString(value);
        String decStr = String.valueOf(value);
        String hexStr = Integer.toHexString(value).toUpperCase();
        // 格式化显示文本（当前进制用【】标注）
        String displayText = String.format(
            "二进制(BIN): %s\n" +
            "八进制(OCT): %s\n" +
            "十进制(DEC): %s\n" +
            "十六进制(HEX): %s",
            currentRadix.equals("BIN") ? "【" + binStr + "】" : binStr,
            currentRadix.equals("OCT") ? "【" + octStr + "】" : octStr,
            currentRadix.equals("DEC") ? "【" + decStr + "】" : decStr,
            currentRadix.equals("HEX") ? "【" + hexStr + "】" : hexStr
        );
        displayArea.setText(displayText); // 更新显示区域
    }
    // 历史记录相关方法：初始化历史记录列表双击复用功能
    private void initHistoryListDoubleClick() {
        historyList.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                // 双击事件（点击次数=2）
                if (e.getClickCount() == 2) {

```

```

        int selectedIndex = historyList.getSelectedIndex(); // 获取选中索引

        if (selectedIndex != -1) {
            // 获取选中的历史记录条目
            String selectedItem =
                historyModel.getElementAt(selectedIndex);
            // 提取表达式（格式：时间 | 表达式 = 结果 → 取" | "和" = "之间的部分）

            String expr = selectedItem.split(" \\| ")[1].split(" = ")[0];

            currentExpr = expr; // 更新当前表达式
            displayArea.setText(currentExpr); // 显示表达式
        }
    }
}

// 加载历史记录：从本地文件 calc_history.txt 读取历史记录
private void loadHistory() {
    File historyFile = new File(HISTORY_FILE);
    if (!historyFile.exists()) {
        return; // 文件不存在：直接返回
    }
    try (BufferedReader reader = new BufferedReader(new
        FileReader(historyFile))) {
        String line;
        // 逐行读取文件内容，添加到历史记录模型
        while ((line = reader.readLine()) != null) {
            historyModel.addElement(line);
        }
    } catch (IOException e) {
        // 读取失败：提示错误信息
        JOptionPane.showMessageDialog(
            this,
            "加载历史记录失败: " + e.getMessage(),
            "错误",
            JOptionPane.ERROR_MESSAGE
        );
    }
}

// 保存历史记录：将计算记录追加到本地文件
private void saveHistory(String record) {
    try (PrintWriter writer = new PrintWriter(new FileWriter(HISTORY_FILE,
        true))) {

```

```

        writer.println(record); // 追加写入记录
    } catch (IOException e) {
        // 写入失败: 提示错误信息
        JOptionPane.showMessageDialog(
            this,
            "保存历史记录失败: " + e.getMessage(),
            "错误",
            JOptionPane.ERROR_MESSAGE
        );
    }
}

// 显示历史记录窗口: 弹出新窗口展示所有历史记录
private void showHistoryWindow() {
    JFrame historyFrame = new JFrame("计算历史记录"); // 历史记录窗口
    historyFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // 关闭
    窗口不退出程序
    historyFrame.setSize(400, 300); // 窗口大小
    historyFrame.setLocationRelativeTo(this); // 相对于主窗口居中
    // 历史记录面板: 边界布局
    JPanel historyPanel = new JPanel(new BorderLayout(5, 5));
    // 标题标签: 显示历史记录格式说明
    JLabel titleLabel = new JLabel("历史记录 (时间 | 表达式 = 结果)");
    titleLabel.setFont(CHINESE_FONT);
    titleLabel.setHorizontalAlignment(JLabel.CENTER);
    historyPanel.add(titleLabel, BorderLayout.NORTH);
    // 无历史记录: 显示提示标签
    if (historyModel.isEmpty()) {
        JLabel emptyLabel = new JLabel("暂无计算历史记录");
        emptyLabel.setFont(CHINESE_FONT);
        emptyLabel.setHorizontalAlignment(JLabel.CENTER);
        historyPanel.add(emptyLabel, BorderLayout.CENTER);
    } else {
        // 有历史记录: 添加带滚动条的列表
        historyPanel.add(new JScrollPane(historyList), BorderLayout.CENTER);
    }
    historyFrame.add(historyPanel); // 添加面板到窗口
    historyFrame.setVisible(true); // 显示窗口
}

// 清空历史记录: 删除本地文件和内存中的历史记录
private void clearHistory() {
    // 弹出确认对话框 (防止误操作)
    int confirm = JOptionPane.showConfirmDialog(
        this,
        "确定要清空所有历史记录吗?",

```

```

        "确认清空",
        JOptionPane.YES_NO_OPTION,
        JOptionPane.WARNING_MESSAGE
    );
    // 用户确认清空
    if (confirm == JOptionPane.YES_OPTION) {
        historyModel.clear(); // 清空内存中的历史记录
        new File(HISTORY_FILE).delete(); // 删除本地文件
        // 提示清空成功
        JOptionPane.showMessageDialog(
            this,
            "历史记录已清空!",
            "提示",
            JOptionPane.INFORMATION_MESSAGE
        );
    }
}
// 工具方法：格式化计算结果（去除末尾.0，过长数字用科学计数法）
private String formatResult(String result) {
    // 结果以.0 结尾：去除末尾的.0（如 10.0→10）
    if (result.endsWith(".0")) {
        return result.substring(0, result.length() - 2);
    }
    // 结果长度超过 16 位且无 e（非科学计数法）：用工程计数法格式化
    if (result.length() > MAX_DIGIT && !result.contains("e")) {
        return new java.math.BigDecimal(result).toEngineeringString();
    }
    return result; // 其他情况：返回原结果
}
// 工具方法：提取表达式中最后一个数字（用于+/-功能）
private String getLastNumber(String expr) {
    StringBuilder num = new StringBuilder();
    // 从表达式末尾向前遍历，提取数字、小数点、负号
    for (int i = expr.length() - 1; i >= 0; i--) {
        char c = expr.charAt(i);
        if (Character.isDigit(c) || c == '.' || c == '-') {
            num.insert(0, c);
        } else {
            break; // 遇到非数字字符：停止提取
        }
    }
    return num.toString();
}
}

```

```

// 工具方法：包装函数表达式（如 sin、cos 等，处理角度单位转换）
private void wrapFunction(String funcName, boolean isTrigonometric) {
    if (currentExpr.isEmpty()) {
        // 表达式为空：直接拼接函数名+左括号（如 sin(）
        currentExpr += funcName + "(";
        // 三角函数且角度制：额外添加左括号（用于角度转换，如 sin((角度)*π/180)）
        if (isTrigonometric && !isRadian) {
            currentExpr += "(";
        }
    } else {
        // 表达式非空：判断最后一个字符是否为运算符或左括号
        char lastChar = currentExpr.charAt(currentExpr.length() - 1);
        if (lastChar == '+' || lastChar == '-' || lastChar == '*' || lastChar
        == '/' || lastChar == '(') {
            currentExpr += funcName + "("; // 直接拼接函数名+左括号
        } else {
            currentExpr += "*" + funcName + "("; // 补乘号后拼接（如
            2sin(x)→2*sin(x)）
        }
        // 三角函数且角度制：额外添加左括号
        if (isTrigonometric && !isRadian) {
            currentExpr += "(";
        }
    }
    displayArea.setText(currentExpr); // 更新显示区域
}

// 工具方法：判断表达式是否包含反三角函数（asin/acos/atan）
private boolean containsInverseTrigFunction(String expr) {
    return expr.contains("Math.asin(")
        || expr.contains("Math.acos(")
        || expr.contains("Math.atan(");
}

// 工具方法：验证反三角函数输入值（必须在[-1, 1]之间）
private boolean validateInverseTrigInput(double input) {
    return input >= -1.0 && input <= 1.0;
}

// ===== 事件监听（整合所有模式） =====
// 初始化所有事件监听器：菜单事件、函数绘图按钮事件
private void initListeners() {
    // 1. 菜单事件监听（模式切换+历史记录操作）
    JMenuBar menuBar = (JMenuBar) topControlPanel.getComponent(0);
    JMenu funcMenu = (JMenu) menuBar.getComponent(0);
    // 遍历"功能"菜单下的所有组件
    for (Component comp : funcMenu.getMenuComponents()) {

```

```

        if (comp instanceof JMenu) {
            // 模式切换子菜单
            JMenu modeMenu = (JMenu) comp;
            for (Component item : modeMenu getMenuComponents()) {
                if (item instanceof JMenuItem) {
                    JMenuItem menuItem = (JMenuItem) item;
                    // 模式切换事件
                    menuItem.addActionListener(e -> {
                        currentMode = menuItem.getText(); // 更新当前模式
                        modeCardLayout.show(modeCardPanel, currentMode); //
切换模式面板

                        JLabel tipLabel = (JLabel)
displayPanel.getComponent(0);
                        currentExpr = ""; // 切换模式：清空当前表达式
                        displayArea.setText(""); // 清空显示区域
                        // 模式专属初始化
                        if (currentMode.equals("程序员模式")) {
                            currentRadix = "DEC"; // 默认十进制
                            tipLabel.setText(
                                "当前模式: 程序员模式 | 当前进制: " + currentRadix
+ " | 支持 32 位整数+位运算 (JS 引擎)"
                            );
                            displayPanel.setVisible(true); // 显示计算面板
                        } else if (currentMode.equals("标准模式")) {
                            tipLabel.setText(
                                "当前模式: 标准模式 | 提示: 通过\"功能\"菜单切换
模式 and 查看历史记录"
                            );
                            displayPanel.setVisible(true);
                        } else if (currentMode.equals("科学模式")) {
                            tipLabel.setText(
                                "当前模式: 科学模式 | 角度单位: " + (isRadian ?
"弧度制" : "角度制") + " | 支持三角函数/指数/对数运算"
                            );
                            displayPanel.setVisible(true);
                        } else if (currentMode.equals("函数绘图模式")) {
                            // 绘图模式: 隐藏计算面板, 显示绘图提示
                            tipLabel.setText("当前模式: 函数绘图模式 | 提示: 输
入表达式后点击\"添加到画布\", 支持多曲线绘制");
                            displayPanel.setVisible(false);
                        }
                    });
                }
            }
        }
    }
}

```

```

        } else if (comp instanceof JMenuItem) {
            // 历史记录相关菜单选项
            JMenuItem menuItem = (JMenuItem) comp;
            if (menuItem.getText().equals("查看历史记录")) {
                // 查看历史记录事件
                menuItem.addActionListener(e -> showHistoryWindow());
            } else if (menuItem.getText().equals("清空历史记录")) {
                // 清空历史记录事件
                menuItem.addActionListener(e -> clearHistory());
            }
        }
    }

    // 2. 函数绘图功能按钮监听（新增）
    addFuncBtn.addActionListener(this); // 添加到画布按钮
    clearFuncBtn.addActionListener(this); // 清空画布按钮
}

// ===== 按钮点击事件处理（整合所有模式） =====
// 按钮点击事件统一处理：根据当前模式分发到对应处理方法
@Override
public void actionPerformed(ActionEvent e) {
    String btnText = ((JButton) e.getSource()).getText();
    // 函数绘图模式的专属按钮处理（新增）
    if ("添加到画布".equals(btnText)) {
        addFunctionToCanvas(); // 调用添加函数方法
        return;
    } else if ("清空画布".equals(btnText)) {
        clearFunctionCanvas(); // 调用清空画布方法
        return;
    }
    // 其他模式的按钮处理（根据当前模式分发）
    if ("标准模式".equals(currentMode)) {
        handleStandardMode(btnText); // 标准模式处理
    } else if ("科学模式".equals(currentMode)) {
        handleScientificMode(btnText); // 科学模式处理
    } else if ("程序员模式".equals(currentMode)) {
        handleProgrammerMode(btnText); // 程序员模式处理
    }
}

// 程序员模式按钮点击处理
private void handleProgrammerMode(String btnText) {
    switch (btnText) {
        case "二进制(BIN)":
            switchRadix("BIN"); // 切换到二进制
            break;
    }
}

```

```

        case "八进制(OCT)":
            switchRadix("OCT"); // 切换到八进制
            break;
        case "十进制(DEC)":
            switchRadix("DEC"); // 切换到十进制
            break;
        case "十六进制(HEX)":
            switchRadix("HEX"); // 切换到十六进制
            break;
        case "C":
            currentExpr = ""; // 清空表达式
            displayArea.setText(""); // 清空显示
            break;
        case "DEL":
            // 删除最后一个字符
            if (!currentExpr.isEmpty()) {
                currentExpr = currentExpr.substring(0, currentExpr.length() -
1);

                displayArea.setText(currentExpr);
            }
            break;
        case "=":
            // 计算表达式
            if (currentExpr.isEmpty()) {
                break;
            }
            try {
                // 使用 JS 引擎计算表达式
                Object resultObj = jsEngine.eval(currentExpr);
                // 转换为 int (32 位位运算)
                int result = ((Number) resultObj).intValue();
                String resultStr = Integer.toString(result);
                // 更新进制显示
                updateProgrammerDisplay(result);
                // 保存到历史记录
                String timeStamp = dateFormat.format(new Date());
                String historyRecord = String.format(
                    "%s | %s = %s",
                    timeStamp,
                    currentExpr,
                    resultStr
                );
                historyModel.addElement(historyRecord);
                saveHistory(historyRecord);
            }
    }
}

```

```

        // 更新 ans 和 currentExpr (供后续操作)
        ans = resultStr;
        currentExpr += " = " + resultStr;
    } catch (ScriptException ex) {
        // 表达式错误: 提示错误信息
        displayArea.setText("表达式错误: " + ex.getMessage());
        currentExpr = "";
    }
    break;
default:
    // 验证输入字符是否符合当前进制
    boolean isValid = false;
    // 根据当前进制验证输入字符合法性
    switch (currentRadix) {
        case "BIN":
            isValid = btnText.matches("[01]"); // 二进制仅允许 0/1
            break;
        case "OCT":
            isValid = btnText.matches("[0-7]"); // 八进制仅允许 0-7
            break;
        case "DEC":
            isValid = btnText.matches("[0-9.]"); // 十进制允许 0-9 和小
            // 数点
            break;
        case "HEX":
            isValid = btnText.matches("[0-9A-Fa-f]"); // 十六进制允许
            // 0-9 和 A-F
            break;
    }
    // 运算符和功能键 (&|^~!<>) 直接允许输入
    if (btnText.matches("[+\\-*/&|^~!<>]")) {
        isValid = true;
    }
    // 输入合法: 拼接表达式并更新显示
    if (isValid) {
        currentExpr += btnText.toUpperCase(); // 字母转为大写 (适配十
        // 六进制)

        displayArea.setText(currentExpr);
    } else {
        // 输入非法: 提示当前进制不支持该字符
        JOptionPane.showMessageDialog(
            this,
            "当前进制不支持该字符!",
            "错误",
            JOptionPane.ERROR_MESSAGE);
    }
}

```

```

        JOptionPane.ERROR_MESSAGE
    );
}

}

// 标准模式按钮点击处理
private void handleStandardMode(String btnText) {
    switch (btnText) {
        case "C":
        case "CE":
            currentExpr = ""; // 清空当前表达式
            displayArea.setText(""); // 清空显示区域
            break;
        case "DEL":
            // 删除表达式最后一个字符
            if (!currentExpr.isEmpty()) {
                currentExpr = currentExpr.substring(0, currentExpr.length() -
1);

                displayArea.setText(currentExpr);
            }
            break;
        case "=":
            calculateResult(); // 调用核心计算方法
            break;
        case "1/x":
            currentExpr += "1/"; // 拼接倒数表达式 (1/xxx)
            displayArea.setText(currentExpr);
            break;
        case "√":
            currentExpr += "Math.sqrt("; // 拼接平方根表达式 (Math.sqrt(xxx))
            displayArea.setText(currentExpr);
            break;
        case "ANS":
            // 拼接上一次计算结果 (ans 变量)
            if (!ans.isEmpty()) {
                currentExpr += ans;
                displayArea.setText(currentExpr);
            }
            break;
        case "+/-":
            // 切换最后一个数字的正负号
            String lastNum = getLastNumber(currentExpr); // 提取最后一个数字
            if (!lastNum.isEmpty()) {

```

```

        int lastNumStart = currentExpr.length() - lastNum.length(); //
        最后一个数字的起始索引
        String sign = lastNum.startsWith("-") ? "" : "-"; // 切换符号
        (无负号→加负号, 有负号→去负号)
        currentExpr = currentExpr.substring(0, lastNumStart)
            + sign
            + lastNum.replaceFirst("-", ""); // 重组表达式
        displayArea.setText(currentExpr);
    }
    break;
case "%":
    currentExpr += "/100"; // 百分比→除以 100 (xxx%→xxx/100)
    displayArea.setText(currentExpr);
    break;
default:
    // 数字输入: 限制最大位数 (16 位)
    if (btnText.matches("[0-9]") &&
        getLastName(currentExpr).length() >= MAX_DIGIT) {
        JOptionPane.showMessageDialog(
            this,
            "数字长度不能超过 16 位!",
            "警告",
            JOptionPane.WARNING_MESSAGE
        );
        return;
    }
    // 拼接其他合法字符 (数字、小数点、运算符、括号)
    currentExpr += btnText;
    displayArea.setText(currentExpr);
}
}

// 科学模式按钮点击处理
private void handleScientificMode(String btnText) {
    // 判断是否需要补全角度转换公式 (角度制时, 三角函数需乘以 $\pi/180$ )
    boolean needCompleteAngleFormula = !isRadian
        && (currentExpr.contains("Math.sin(")
            || currentExpr.contains("Math.cos(")
            || currentExpr.contains("Math.tan(")
            || currentExpr.contains("Math.atan("));

    // 判断函数名后是否直接跟数字 (如 sin3→sin(3))
    boolean isFunctionFollowedByNumber = btnText.matches("[0-9.]")
        && !currentExpr.isEmpty()
        && (currentExpr.endsWith("sin")

```

```

        || currentExpr.endsWith("cos")
        || currentExpr.endsWith("tan")
        || currentExpr.endsWith("cot")
        || currentExpr.endsWith("asin")
        || currentExpr.endsWith("acos")
        || currentExpr.endsWith("atan")
        || currentExpr.endsWith("log")
        || currentExpr.endsWith("ln")
        || currentExpr.endsWith("sqrt")
        || currentExpr.endsWith("cbrt")
        || currentExpr.endsWith("exp"));
// 函数名后直接跟数字：自动补左括号（如 sin3→sin(3)
if (isFunctionFollowedByNumber) {
    currentExpr += "(" + btnText;
    displayArea.setText(currentExpr);
    return;
}
switch (btnText) {
    case "C":
    case "CE":
        currentExpr = ""; // 清空表达式
        displayArea.setText(""); // 清空显示
        break;
    case "DEL":
        // 删除最后一个字符
        if (!currentExpr.isEmpty()) {
            currentExpr = currentExpr.substring(0, currentExpr.length() -
1);

            displayArea.setText(currentExpr);
        }
        break;
    case "=":
        calculateResult(); // 调用核心计算方法
        break;
    case "Rad":
        isRadian = true; // 切换为弧度制
        JOptionPane.showMessageDialog(
            this,
            "已切换为弧度制",
            "提示",
            JOptionPane.INFORMATION_MESSAGE
        );
        // 更新提示标签
        ((JLabel) displayPanel.getComponent(0)).setText(

```

```

        "当前模式：科学模式 | 角度单位：弧度制 | 支持三角函数/指数/对数运
算"

        );
        break;
    case "Deg":
        isRadian = false; // 切换为角度制
        JOptionPane.showMessageDialog(
            this,
            "已切换为角度制",
            "提示",
            JOptionPane.INFORMATION_MESSAGE
        );
        // 更新提示标签
        ((JLabel) displayPanel.getComponent(0)).setText(
            "当前模式：科学模式 | 角度单位：角度制 | 支持三角函数/指数/对数运
算"

        );
        break;
    case "π":
        currentExpr += Math.PI; // 拼接π常量 (Math.PI)
        displayArea.setText(currentExpr);
        break;
    case "e":
        currentExpr += Math.E; // 拼接自然常数 e (Math.E)
        displayArea.setText(currentExpr);
        break;
    case "sin":
        wrapFunction("Math.sin", true); // 包装正弦函数 (处理角度单位)
        break;
    case "cos":
        wrapFunction("Math.cos", true); // 包装余弦函数 (处理角度单位)
        break;
    case "tan":
        wrapFunction("Math.tan", true); // 包装正切函数 (处理角度单位)
        break;
    case "cot":
        wrapFunction("Math.atan(Math.PI/2 - ", true); // 余切函数→atan(π/2
- x)

        break;
    case "asin":
        wrapFunction("Math.asin", false); // 包装反正弦函数
        break;
    case "acos":
        wrapFunction("Math.acos", false); // 包装反余弦函数

```

```

        break;
    case "atan":
        wrapFunction("Math.atan", false); // 包装反正切函数
        break;
    case "log":
        wrapFunction("Math.log10", false); // 包装常用对数 (log10)
        break;
    case "ln":
        wrapFunction("Math.log", false); // 包装自然对数 (ln)
        break;
    case "sqrt":
        wrapFunction("Math.sqrt", false); // 包装平方根函数
        break;
    case "cbrt":
        wrapFunction("Math.cbrt", false); // 包装立方根函数
        break;
    case "exp":
        wrapFunction("Math.exp", false); // 包装指数函数 (e^x)
        break;
    case "x^2":
        currentExpr += "**2"; // 平方运算→**2 (JS 语法)
        displayArea.setText(currentExpr);
        break;
    case "x^y":
        currentExpr += "**"; // 幂运算→** (JS 语法)
        displayArea.setText(currentExpr);
        break;
    case ")":
        // 补全括号: 角度制时需添加角度转换公式的右括号
        if (needCompleteAngleFormula && currentExpr.endsWith("(")) {
            currentExpr += ")"; // 表达式末尾为左括号→直接补右括号
        } else if (needCompleteAngleFormula) {
            currentExpr += ") * Math.PI / 180"; // 补角度转换公式 (*π/180)
        } else {
            currentExpr += ")"; // 其他情况→直接补右括号
        }
        displayArea.setText(currentExpr);
        break;
    default:
        // 拼接其他字符 (数字、小数点、运算符等)
        currentExpr += btnText;
        displayArea.setText(currentExpr);
}
    
```

和右括号


```

        displayArea.setText("反三角函数输入错误：请输入有效的数值");
        currentExpr = "";
        return;
    }
}
// 执行表达式计算
Object resultObj = jsEngine.eval(expr);
double result = Double.parseDouble(resultObj.toString());
String resultStr = "";
// 科学模式+角度制+反三角函数：结果转换为角度（弧度→角度）
if ("科学模式".equals(currentMode) && !isRadian && isInverseTrig) {
    double angleResult = result * 180 / Math.PI;
    resultStr = formatResult(String.valueOf(angleResult)) + "°"; // 拼
接角度符号
} else {
    resultStr = formatResult(String.valueOf(result)); // 普通结果格式
化
}
// 格式化显示文本（原始表达式 + 补全的括号 + 结果）
String fullExpr = originalExpr
    + (openBrackets > 0 ? ")" : "")
    + "=\n"
    + resultStr;
displayArea.setText(fullExpr); // 更新显示区域
// 保存到历史记录（去除角度符号，避免影响复用）
String historyResult = resultStr.replace("°", "");
String timeStamp = dateFormat.format(new Date()); // 时间戳
String historyRecord = timeStamp
    + " | "
    + originalExpr
    + (openBrackets > 0 ? ")" : "")
    + " = "
    + historyResult;
historyModel.addElement(historyRecord); // 添加到内存历史记录
saveHistory(historyRecord); // 持久化到本地文件
// 更新当前表达式和上一次结果（供后续操作）
currentExpr = historyResult;
ans = currentExpr;
} catch (ScriptException e) {
    // 表达式计算错误：分类提示错误信息
    String errorMsg = e.getMessage();
    if (errorMsg.contains("Infinity") || errorMsg.contains("NaN")) {
        displayArea.setText("计算错误：除零或无效输入");
    } else if (errorMsg.contains("Unexpected token")) {

```

```

        displayArea.setText("表达式错误: 语法格式不正确");
    } else {
        displayArea.setText("表达式错误: " + errorMsg);
    }
    currentExpr = ""; // 清空错误表达式
}
}
// ===== 主方法 =====
// 程序入口: 初始化并启动计算器应用
public static void main(String[] args) {
    try {
        // 设置系统外观 (与操作系统风格一致)
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {
        e.printStackTrace(); // 外观设置失败→打印异常栈
    }
    // 在 EDT (事件调度线程) 中启动应用 (Swing 线程安全要求)
    SwingUtilities.invokeLater(Calculate_v5::new);
}
}

```

(3) 实验过程

1. 函数绘图 UI 设计

一开始, 我理想中的函数绘图模式是基于键盘输入来实现的, 考虑到由于用户的输入可能会对函数带来非常多的不确定性, 不一定能正确进行运算, 为操作的简便性带来挑战, 因此, 我考虑复用科学模式下的各种函数, 同时为了让面板不那么臃肿, 我们删去了数字按键, 改为键盘输入。

对于函数的画布, 我们想通过 JPanel 来实现, 通过重写 paintComponent, Graphics2D 类, 为其设置好背景色与网格后, 就是一块标准的画布了:

```

funcCanvasPanel = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g); // 调用父类绘制 (确保背景等正常显示)
        Graphics2D g2d = (Graphics2D) g; // 转换为 2D 绘图对象
        // 开启抗锯齿 (使线条、文本更平滑)
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        int canvasWidth = getWidth(); // 画布宽度
        int canvasHeight = getHeight(); // 画布高度
    }
}

```

```

        // 绘制白色画布背景（避免与网格/坐标轴混淆）
        g2d.setColor(Color.WHITE);
        g2d.fillRect(AXIS_MARGIN, AXIS_MARGIN, canvasWidth - 2 *
AXIS_MARGIN, canvasHeight - 2 * AXIS_MARGIN);
        // 绘制网格线
        drawGrid(g2d, canvasWidth, canvasHeight);
        // 绘制坐标轴（含箭头、刻度、标签）
        drawAxis(g2d, canvasWidth, canvasHeight);
        // 绘制所有已添加的函数曲线
        drawAllFunctions(g2d, canvasWidth, canvasHeight);
        // 绘制图例（右上角，显示函数表达式和颜色）
        drawLegend(g2d, canvasWidth);
    }
};
funcCanvasPanel.setBackground(Color.LIGHT_GRAY); // 画布背景色（浅灰色）
// 组装函数绘图主面板：输入面板（北）+ 绘图画布（中）
funcPlotPanel.add(funcInputPanel, BorderLayout.NORTH);
funcPlotPanel.add(funcCanvasPanel, BorderLayout.CENTER);
return funcPlotPanel;
}

```

运行代码，观察具体效果：

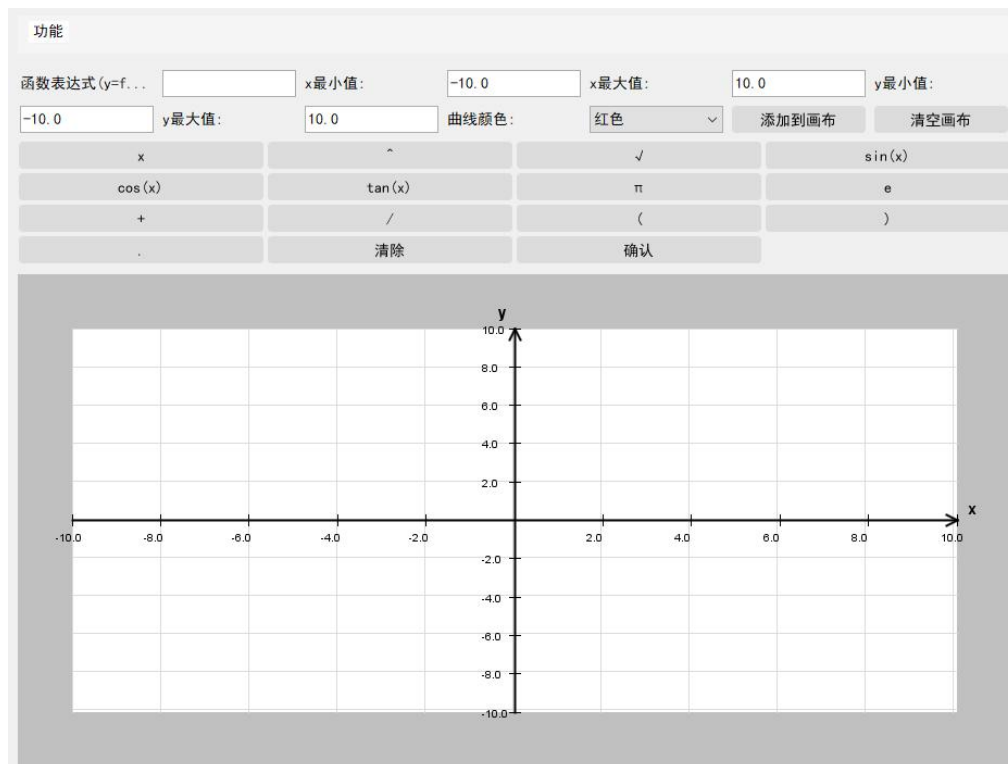


图 25 函数绘图 UI 设计

可以发现，我们的 UI 设计已经完成得差不多了，我们开始对于数据处理功能的逻辑

编写。

2. 绘图逻辑实现

参考 Matlab 的绘图功能，其是基于打点连线来实现绘图功能，同样的，参照 CSDN 上一个博客，我们也能得到画点的方法示例，因此，我们需要做的工作在于将输入的函数表达式转化为可以被计算的表达式，同时确定函数的定义域与值域，同时，根据函数表达式确定一系列点的坐标，再根据坐标判断该点是否应该被绘制，由此，我们可以完成这部分代码的编写了。

核心逻辑代码如下：

```
private void drawAllFunctions(Graphics2D g2d, int canvasWidth, int
canvasHeight) {
    // 采样点数：确保曲线平滑（最小 2000 点，或根据画布宽度自适应）
    int sampleCount = Math.max(2000, (canvasWidth - 2 * AXIS_MARGIN) / 2);
    // x 轴采样步长：均匀分布在 xMin 到 xMax 之间
    double xStep = (xMax - xMin) / (sampleCount - 1);
    // 记录绘制失败的函数（用于后续提示）
    List<String> failedFuncs = new ArrayList<>();

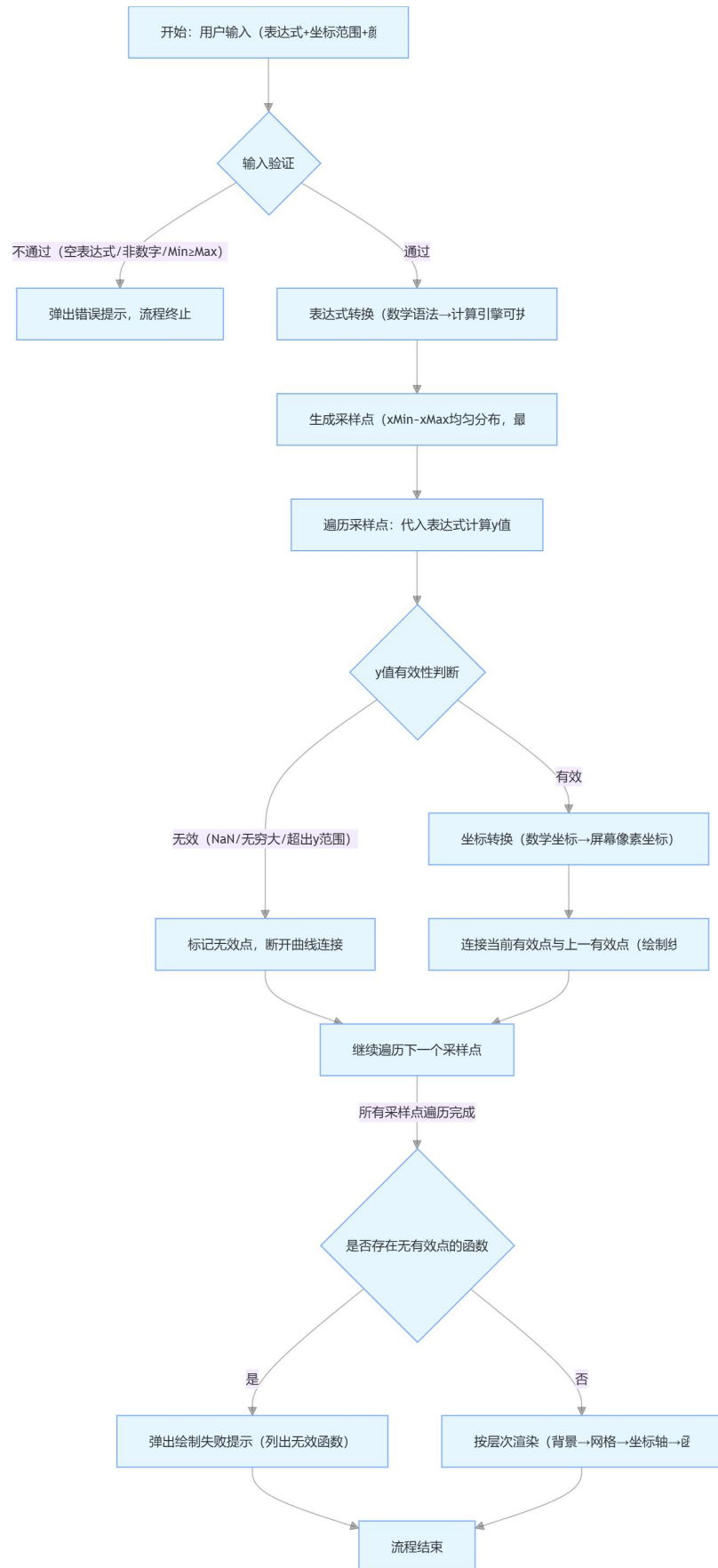
    // 遍历每个已添加的函数
    for (FuncData func : funcList) {
        // 设置当前函数曲线颜色
        g2d.setColor(func.color);
        g2d.setStroke(new BasicStroke(2)); // 曲线线宽（2 像素）
        Point prevPoint = null; // 上一个有效采样点（用于连线）
        // 有效采样点计数（判断函数是否可绘制）
        int validCount = 0;
        // 遍历所有采样点，计算并绘制
        for (int i = 0; i < sampleCount; i++) {
            double x = xMin + i * xStep; // 当前采样点的 x 值（数学坐标系）
            double y = calculateFuncY(func.expr, x); // 计算对应的 y 值
            // 验证 y 值有效性：非 NaN、非无穷大、在 y 轴范围内
            if (!Double.isNaN(y) && !Double.isInfinite(y) && y >= yMin && y <=
yMax) {
                validCount++;
                // 转换为屏幕像素坐标
                int pixelX = xToPixel(x, canvasWidth);
                int pixelY = yToPixel(y, canvasHeight);
                Point currentPoint = new Point(pixelX, pixelY);
                // 连接当前点和上一个有效点（绘制线段）
                if (prevPoint != null) {
```

```

        g2d.draw(new Line2D.Double(prevPoint.x, prevPoint.y,
currentPoint.x, currentPoint.y));
    }
    // 更新上一个有效点
    prevPoint = currentPoint; // 更新上一个有效点
} else {
    // 无效点，断开曲线（避免绘制错误线段）
    prevPoint = null;
}
}
// 无有效点：记录该函数绘制失败
if (validCount == 0) {
    failedFuncs.add("y = " + func.expr);
}
}
// 提示绘制失败的函数（通过 SwingUtilities 确保在 EDT 线程执行）
if (!failedFuncs.isEmpty()) {
    SwingUtilities.invokeLater(() -> {
        String msg = "以下函数在当前坐标范围无法绘制，请检查表达式或调整 x/y
范围：\n\n";
        msg += String.join("\n", failedFuncs);
        JOptionPane.showMessageDialog(this, msg, "绘制失败",
JOptionPane.WARNING_MESSAGE);
    });
}
}

```

其中，处理逻辑可以以如下流程图描述：



验证功能是否完善，我们可以通过画一个简单的图像：

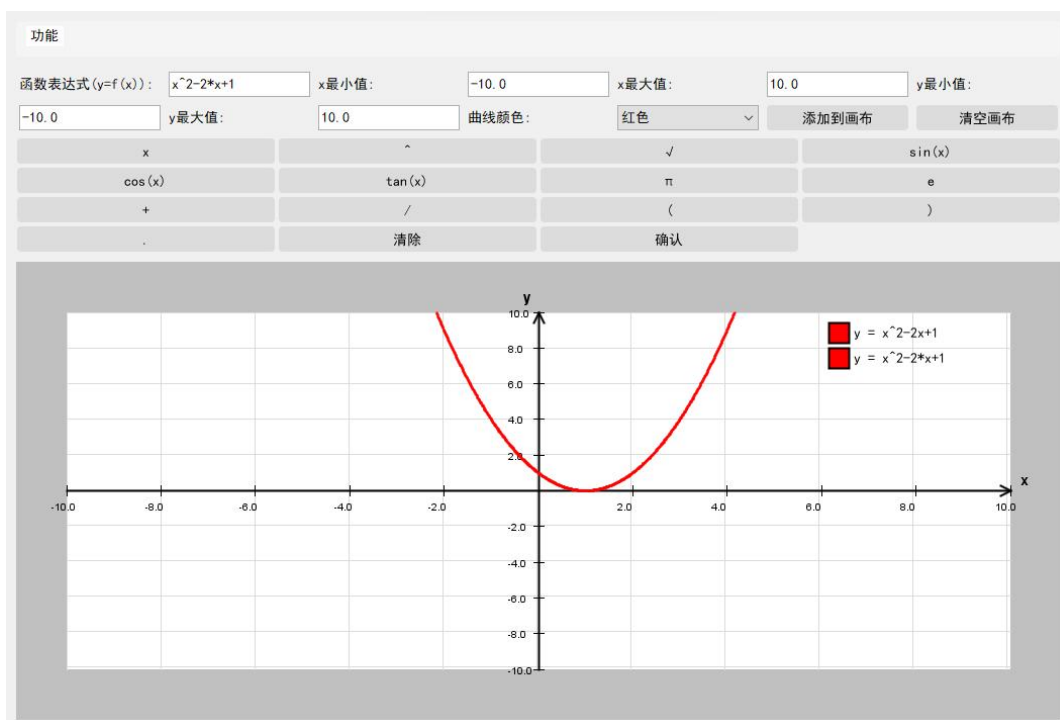


图 26 简单函数绘制逻辑测试

可以发现，我们的功能构建是正确的。

【实验感想】

本次 Java 编程技术大作业 —— 计算器的实现，是一次从理论到实践的完整探索之旅，让我对 Java 编程尤其是 Swing GUI 开发有了更为深刻的理解与掌握。从最初仅能实现基础四则运算的简易计算器，到最终集成标准、科学、程序员、函数绘图四大模式及历史记录功能的综合工具，我一次次在查阅相关资料，根据目标功能进行的每一次迭代都充满了挑战与收获。

在一开始，我仅聚焦于核心计算逻辑与基础 UI 搭建，实现了一个非常简易的计算器，但对比学长的计算器成品，我的计算器明显布局混乱、按钮排列不美观。我通过切换更灵活的布局管理器、自定义组件样式（重写方法），我学会了如何运用 BorderLayout、GridBagLayout 等布局工具优化界面结构，并将他们协同运用，为我的 UI 界面添砖加瓦，也学习了如何使用 Graphics2D 类来绘制样式，更理解了“前端设计需兼顾美观与实用性”的重要性。为实现圆角按钮与鼠标悬浮交互效果，我尝试继承

JButton 并重写 paintComponent 方法，这个过程让我深入体会到 Java 面向对象编程中继承与多态的灵活运用，也明白了自定义组件时细节处理（如颜色初始化、抗锯齿设置）对最终效果的影响。

功能拓展阶段是本次实验的核心难点。当我试图实现科学模式的角度制与弧度制切换时，因误用 JS 引擎不支持的语法导致频繁报错，最终我巧妙的通过监听括号输入并手动补全转换公式的方式解决了问题；开发函数绘图功能时，从坐标转换、采样点计算到曲线绘制，每一步都需要精准的逻辑推导，尤其是将数学坐标系映射为屏幕像素坐标系的过程，让我切实感受到编程与数学的紧密关联。而历史记录的本地持久化实现，则让我掌握了 Java 文件读写的基本操作，理解了数据持久化的核心思路。

整个开发过程中，我深刻体会到“迭代开发”与开源项目的价值。从最初的功能残缺到逐步补全退格键、ANS 复用、表达式自动补全，再到优化用户交互体验，每一次改进都源于对“用户需求”的深入思考。这其中经历的崎岖道路绝不仅仅是在报告中展示的那么轻易，每一个问题的解决都耗费了我大量的精力，幸好，通过参考开源项目、查阅技术文档，让我得以更加高效地解决问题。通过这次大作业，我学会了如何高效利用资源，它培养了我独立解决问题的能力。

此次实验不仅巩固了我对 Java Swing、事件监听、异常处理等核心知识点的掌握，还让我知道，一个成功的项目不仅需要实现功能，更要兼顾代码的可读性、扩展性与用户体验。未来，我希望能进一步优化该计算器，添加更多实用功能，同时在代码结构优化、性能提升等方面继续探索，不断提升自己的编程能力。

【参考文献】

- [1]. 冯大辉, Java Swing 圆角按钮的实现与优化, [Java Swing 圆角按钮的实现与优化 - 姚伟斌](#)
- [2]. Java 实现计算器, [java:实现计算器（附带源码）_java 计算器引擎-CSDN 博客](#)
- [3]. Java 实现函数绘制, [Java 绘制数学函数图像-CSDN 博客](#)